

Lecture 4 — Refactoring & Maintainable Code

Lecture id: L04 **Source decks:** Refactoring1.pdf (59 p — Fowler's refactoring catalog), BetterCode.pdf (58 p — "Building Maintainable Software", the 10 SIG guidelines / BetterCodeHub), HighLevelRefactoring.pdf (22 p — Kerievsky's *Refactoring to Patterns*, high-level/composite refactorings) **Labs:** RefactoringLab1.pdf ([RefactLab] — find code smells in JHotDraw with SonarLint, apply Kerievsky refactorings on a feature branch) **Process phase(s):** Prefactoring · Postfactoring (the two refactoring bookends of Rajlich's change process) **Citation key:** (Refactoring1 p.X), (BetterCode p.X), (HighLevelRefactoring p.X), (RefactLab p.1); readings [Fowler99] (Fowler, *Refactoring*, 1999), [Ker05] (Kerievsky, *Refactoring to Patterns*, 2005), [VRvdL+16] (Visser et al. / SIG, *Building Maintainable Software*, 2016), [MC09] (Martin & Coplien, *Clean Code*, 2009), [Raj13] (Rajlich, *Software Engineering: The Current Practice*), [GHJV94] (Gang of Four, *Design Patterns*), [Martin] (Martin, *Clean Code*). **Grounding note:** Every threshold, smell name, and refactoring name below is taken from the three decks. Where a slide is image-only (the BetterCode "Level / Measure / Example" pages and the HighLevelRefactoring pattern-structure pages), the page was rendered and read directly; those claims carry a (BetterCode p.X) citation. Connections to Rajlich's change process and to JHotDraw that are *not literally on a slide* (the decks teach the techniques generically) are flagged as **[course-context inference]** so you can distinguish slide-fact from study-scaffolding. Page numbers in parentheses inside refactoring names (e.g. "Extract Method (110)") are the page numbers in the *source book* ([Fowler99] or [Ker05]) exactly as printed on the slides — they are NOT slide numbers.

Overview

Lecture 4 is the **craft** lecture of the course: it teaches *how* to change code well, not *where* to change it (that was Concept Location, L02/L03) or *what* will be affected (Impact Analysis). It sits on top of two phases of Rajlich's eight-phase change process — **Prefactoring** (clean up the code *before* you inject the new functionality, so the change becomes easy) and **Postfactoring** (clean up *after*, so you don't leave the code worse than you found it) **[course-context inference; the decks define refactoring generically, the course maps it onto these two phases]**.

The reason refactoring earns a whole lecture is that software does not rot on its own — it rots because people change it. Each feature, bug-fix, and quick hack adds a little disorder; left unmanaged, the design degrades until even small changes become expensive and risky (this is the practical face of "technical debt"). Refactoring is the *deliberate, behaviour-preserving counter-force* that repays that debt: it keeps the internal structure clean enough that the system stays cheap to keep changing. So the lecture's value is economic as much as technical — it is about keeping a codebase *maintainable* over its whole life.

The lecture is taught from three complementary sources, each answering a different question about the same activity:

1. **Refactoring1 (Fowler)** — the canonical *low-level* catalog. **What it is:** the foundational vocabulary of refactoring — the named, mechanical, behaviour-preserving moves and the smells that trigger them. **What it's for:** giving you a shared, precise language ("apply *Extract Method* here") and a recognisable list of warning signs, so refactoring is repeatable rather than ad-hoc. It answers three questions — *What is refactoring?* (a behaviour-preserving change to internal structure) (Refactoring1 p.3), *Why/when*

refactor? (Refactoring1 p.4–5), and *which smells trigger which refactoring?* It enumerates **22 code smells** ("Symptoms of Bad Code") (Refactoring1 p.6–8) and a catalog of ~50 named refactorings grouped into **7 categories** (Refactoring1 p.9).

2. **BetterCode (SIG / "Building Maintainable Software")** — the *measurable* layer. **What it is:** ten guidelines, each with a hard, countable metric threshold, drawn from the Software Improvement Group (SIG) maintainability model and operationalised by **bettercodehub.com** (BetterCode p.3). **What it's for:** turning a subjective judgement ("this code smells bad") into an objective, automatable verdict ("this unit is 23 lines, the threshold is 15, so split it"). **When/how applied:** a tool measures the codebase, flags units/modules/components that exceed thresholds, and you refactor until each is back under threshold — so the same activity Fowler describes qualitatively becomes a pass/fail gate you can put in CI.
3. **HighLevelRefactoring (Kerievsky)** — the *high-level / composite* layer. **What it is:** refactoring to design patterns: a smell is resolved by a *sequence* of small Fowler-style steps that, together, introduce a known design pattern. **What it's for:** giving you bigger, design-aware moves and a thought process for reaching good designs gradually instead of guessing them up front. "Design patterns are the word problems of programming; refactoring is its algebra." (HighLevelRefactoring p.8–9).

The throughline: **refactoring is behaviour-preserving restructuring** that you apply continuously, measured against objective thresholds, to keep a system maintainable enough to keep changing it. The observable behaviour must never change — only the internal structure (Refactoring1 p.3). Read the three decks as three altitudes of one activity: Fowler is the *alphabet* (atomic moves), Kerievsky is the *grammar* (composing moves into patterns), and SIG/BetterCode is the *ruler* (objective limits telling you when a move is needed).

How the three decks cite each other (cross-deck wiring)

The decks are not three parallel monologues — they explicitly reference one another's catalogs, and the exam can test that wiring:

- **BetterCode names Fowler refactorings as its fixes.** The G1 "Refactorings" slide lists exactly **Extract Method (110)** and **Replace Method with Method Object (135)** (BetterCode p.11) — the same names *and the same* [Fowler99] book pages as the Refactoring1 slides (Refactoring1 p.11, 13). The G2 "Refactorings" slide lists **Replace Conditional with Polymorphism pattern (255)** (BetterCode p.18) — again identical to Refactoring1 p.35. So SIG's metric gates dispatch you straight back into Fowler's catalog.
 - **Kerievsky's smell table marks Fowler's smells with [F]** — Alternative Classes with Different Interfaces, Duplicated Code, Large Class, Lazy Class, Long Method, Primitive Obsession, Switch Statements all carry the [F] flag, meaning "this smell is in Fowler's 22 too; here is its *pattern-level* fix" (HighLevelRefactoring p.12–13).
 - **The lab cites both catalogs:** smell descriptions come from **Chapter 4 of [Ker05]**, applied refactorings are "**from [Ker05]**", and the footnoted reference catalog is **refactoring.com/catalog** — Fowler's online catalog (RefactLab p.1).
 - **Same name, different book page — a deliberate trap:** two refactorings exist in *both* catalogs under one name but different page numbers, because they appear in both books. **Introduce Null Object (260)** in Fowler (Refactoring1 p.36) but **(301)** in Kerievsky (HighLevelRefactoring p.12); **Form Template Method** is **(345)** in Fowler (Refactoring1 p.47) but **(205)** in Kerievsky (HighLevelRefactoring p.12). If a question quotes a page number with the name, the number tells you *which book/deck* is being cited.
-

Learning Objectives

After this lecture you should be able to:

1. **Define refactoring** precisely (behaviour-preserving internal-structure change) and state why the *observable behaviour must not change* (Refactoring1 p.3).
2. **Justify** when and why to refactor — the four "why" reasons and the four "when" triggers, including the **Rule of Three** (Refactoring1 p.4–5).
3. **Recognise all 22 code smells** by name and definition, and pick an appropriate refactoring for each (Refactoring1 p.6–8; HighLevelRefactoring p.12–13).
4. **Name and apply the catalog refactorings** in their 7 categories — Composing Methods, Moving Features, Organizing Data, Simplifying Conditionals, Making Method Calls Simpler, Dealing with Generalization, Big Refactorings (Refactoring1 p.9–58).
5. **State the 10 SIG maintainability guidelines and their numeric thresholds** (15 LOC, ≤ 4 branch points, no duplication ≥ 6 lines, ≤ 4 parameters, etc.) and map each to its level — Unit / Module / Component / System (BetterCode p.5–58).
6. **Explain high-level (composite) refactoring** — refactoring *toward* patterns, the smell→pattern catalog, the idea of "the algebra of refactoring," and the difference between high-level and low-level refactoring (HighLevelRefactoring p.8–22).
7. **Distinguish Prefactoring from Postfactoring** and place both in Rajlich's change process (Initiation → Concept Location → Impact Analysis → **Prefactoring** → Actualization → **Postfactoring** → Conclusion → Verification) [**course-context inference**].
8. **Carry out the lab workflow** — feature branch, SonarLint, identify smells in JHotDraw, apply Kerievsky refactorings, document the strategy (RefactLab p.1).

Key Concepts

What refactoring is (the definition)

Refactoring: "A change made to the internal structure of software to make it easier to understand, and cheaper to modify. **The observable behavior of the software should not be changed.**" (Refactoring1 p.3)

What it is. Refactoring is the disciplined reorganisation of *how* code is written without touching *what* it does. You move, rename, split, and merge structure (methods, classes, fields, conditionals) so the program is clearer, while every externally observable effect — return values, side effects, persisted state, thrown exceptions — stays identical.

What it's used for. Two halves matter equally. (a) The *goal* is human-facing — *understandability* and *modifiability* (cheaper future change), not faster runtime, not new features. You refactor so the *next* developer (often future-you) can read and change the code with less effort and less risk. (b) The *constraint* is behaviour preservation: same inputs → same outputs, same side effects. This is what makes refactoring *safe* and what makes automated tooling (and tests) able to verify it. The lab restates this: "altering its internal structure without changing its external behavior... a series of small **behavior preserving transformations**" (RefactLab p.1).

When & how it's applied. A single refactoring "does little, but a sequence of these transformations can produce a significant restructuring. Since each refactoring is small, it's less likely to go wrong. The system is kept fully working after each refactoring" (RefactLab p.1). Concretely: you make one tiny named move (e.g. extract three lines into a well-named method), run the tests, commit, then make the next move — never a big-bang rewrite. This is the **discipline of small steps** — the same idea Kerievsky calls "the algebra of refactoring" (HighLevelRefactoring p.10). **Tests are the safety net:** behaviour preservation is *verified*, not assumed — which is why BetterCode Guideline 9 is "Automated Tests" (BetterCode p.53) and why the course's Verification phase pairs with refactoring [**course-context inference**]. In the JHotDraw lab this looks like: branch, let SonarLint flag a smell, apply one catalog move, re-run tests, confirm the drawing editor still behaves identically, repeat.

Why refactor? (four reasons)

(Refactoring1 p.4)

These are the four justifications you give when someone asks "why spend time changing code that already works?" Each names a benefit that pays off over the life of the system:

1. **Refactoring Improves the Design of Software** — *what/why*: design naturally decays under a stream of changes, because each change is made locally without re-thinking the whole. Refactoring periodically restores the design (removes duplication, re-homes responsibilities) so the structure keeps reflecting what the system actually does. Without it, the design slowly stops matching the code.
2. **Refactoring Makes Software Easier to Understand** — *what/why*: code is read far more often than it is written, so optimising for the reader pays back many times. A good name or a small extracted method turns "what does this block do?" into something self-evident, cutting the time every future maintainer spends decoding it.
3. **Refactoring Helps You Find Bugs** — *what/why*: the act of clarifying structure forces you to understand the code, and that understanding surfaces hidden defects (a missed case, an off-by-one, a swallowed exception) that were invisible inside the tangle. Clean structure makes defects stand out.
4. **Refactoring Helps You Program Faster** — *what/why*: the counter-intuitive but central claim. Bad design slows every future change; good design lets you add features quickly. So the time spent refactoring is repaid by faster subsequent development — refactoring is an *investment in velocity*, not a tax on it.

When to refactor? (four triggers + the Rule of Three)

(Refactoring1 p.5)

These are the concrete moments that should prompt a refactoring — the point being that refactoring is *opportunistic and continuous*, woven into normal work, not a separate scheduled phase:

- **The Rule of Three:** *what it is* — a heuristic for when duplication becomes worth removing. "Refactor the third time you do something similar." (First time, just do it. Second, wince but duplicate. Third, refactor.) *What it's for* — it stops you over-engineering on the first occurrence (you may never see a second) while not letting duplication metastasise; the third repetition is statistically when a pattern is real and abstraction pays off. This is the canonical antidote to Duplicated Code.
- **Refactor When You Add Function** — *what/why*: before slotting in a new feature, clean up the area so the new code fits cleanly instead of being bolted onto a mess. (This is **Prefactoring** in change-process terms [**course-context inference**].)
- **Refactor When You Need to Fix a Bug** — *what/why*: if a bug is hard to find, it is often because the code is unclear; refactoring the area until it is readable makes the bug visible, and the cleaner code resists

similar bugs in future.

- **Refactor As You Do a Code Review** — *what/why*: reviews are where a second pair of eyes surfaces smells; acting on them while the context is fresh keeps quality from drifting. (This is **Postfactoring** territory [**course-context inference**].)

Prefactoring vs Postfactoring (the change-process framing)

The decks themselves teach refactoring generically; the course splits it across two change-process phases [**course-context inference based on Rajlich's process named in the brief**]. The key insight is that *the same techniques* serve two different jobs depending on *where in a change* you apply them:

- **Prefactoring** — *what it is*: refactoring done *before* Actualization. You have located the concept and analysed impact; the code in the change region is tangled, so you clean it up *first*. *What it's for*: to turn a hard change into an easy one — you reshape the existing structure so the new behaviour has an obvious, clean place to live. Maps to Fowler's "Refactor When You Add Function" (Refactoring1 p.5). *When/how*: e.g. before adding a new payment method, you first *Extract Method / Replace Conditional with Polymorphism* on the existing payment switch so the new case becomes a one-class addition. **No new functionality is added in this phase** — it is pure restructuring, behaviour-preserving.
- **Postfactoring** — *what it is*: refactoring done *after* Actualization. The new code works but may have introduced smells (duplication, a long method, a fat class). *What it's for*: to remove the debt the change just created before concluding, so the system is left at least as clean as you found it (the Boy-Scout rule, [Martin]). *When/how*: after the feature passes, you extract the duplication it introduced, shorten the method that grew too long, and re-measure against the SIG thresholds. Maps to BetterCode Guideline 10 "leave no code smells behind after development work" (BetterCode p.57).

Both are behaviour-preserving; the difference is *position relative to the functional change*, not technique. The same catalog (Fowler/Kerievsky) and the same metrics (SIG) apply to both. The mental model: Prefactoring *makes room* for the change; Actualization *makes the change*; Postfactoring *cleans up after* the change.

The 22 Code Smells ("Symptoms of Bad Code")

What a smell is. A code smell is a *surface symptom* — a recognisable pattern in the source that usually (not always) signals a deeper design problem and points to a candidate refactoring (Refactoring1 p.6–8). Smells are *not bugs*: the program may run perfectly while still smelling. Their value is as a *trigger language* — naming a smell tells you both what to look for and, usually, which refactoring to reach for. The full list of 22 follows; for each, note *what it looks like* and *which refactoring fixes it*.

Set 1 (Refactoring1 p.6):

1. **Duplicated Code** — *what*: the same structure appears in more than one place. *Why it matters*: every fix or change must now be made in several places, and one will inevitably be missed → bugs. *Fix*: Extract Method (pull the shared code into one method), Extract Superclass / Form Template Method (when duplicated across sibling classes), Pull Up Method.
2. **Long Method** — *what*: a method that is too long, doing many things at different levels of abstraction. *Why it matters*: long methods are hard to read, test, and reuse, and tend to hide duplication. *Fix*: Extract Method (the workhorse), Replace Method with Method Object (when locals are too entangled to extract directly).
3. **Large Class** — *what*: a class doing too much (too many fields/methods), a "god class". *Why it matters*: it violates single-responsibility, is hard to understand, and changes for many unrelated reasons. *Fix*: Extract Class, Extract Subclass, Extract Interface.
4. **Long Parameter List** — *what*: a method takes too many parameters. *Why it matters*: long parameter lists are hard to call correctly and hard to remember; arguments often travel together as a hidden concept. *Fix*: Introduce Parameter Object, Replace Parameter with Method, Preserve Whole Object.
5. **Divergent Change** — *what*: "when one class is commonly changed in different ways for different

reasons." (One class, many unrelated reasons to change → violates single-responsibility.) *Fix*: Extract Class to split the responsibilities so each changes for one reason. 6. **Shotgun Surgery** — *what*: "when every time you make a kind of change, you have to make a lot of little changes to a lot of different classes." (One change, many classes — the inverse of Divergent Change.) *Why it matters*: the responsibility is scattered, so changes are error-prone and easy to do incompletely. *Fix*: Move Method / Move Field to gather the scattered behaviour into one class. 7. **Feature Envy** — *what*: "a method that seems more interested in a class other than the one it actually is in" — it calls many methods/fields of another object. *Why it matters*: behaviour lives apart from the data it uses, increasing coupling. *Fix*: Move Method (to the class it envies), or Extract Method then Move Method. 8. **Data Clumps** — *what*: "bunches of data that regularly appear together" (the same group of fields/parameters recurring). *Why it matters*: the recurring group is really one missing concept; repeating it spreads change. *Fix*: Extract Class (for fields) / Introduce Parameter Object (for parameters).

Set 2 (Refactoring1 p.7): 9. **Primitive Obsession** — *what*: "excessive use of primitives, due to reluctance to use small objects for small tasks" (using a `String` for a phone number, an `int` for money). *Why it matters*: validation and behaviour that belong to the concept get scattered as primitive-fiddling. *Fix*: Replace Data Value with Object, Replace Type Code with Class/Subclasses. 10. **Switch Statements** — *what*: type-switching logic (`switch` / `if` chains on a type code) that should be polymorphism, especially when the same switch recurs. *Why it matters*: adding a new case means editing every switch — open/closed violation. *Fix*: Replace Conditional with Polymorphism (Fowler) / Replace Conditional Dispatcher with Command or Replace Conditional Logic with Strategy (Kerievsky). 11. **Parallel Inheritance Hierarchies** — *what*: "where every time you make a subclass of one class, you also have to make a subclass of another." *Why it matters*: a special case of Shotgun Surgery — the two hierarchies must be kept in lock-step. *Fix*: Move Method/Field to merge, ultimately Tease Apart Inheritance. 12. **Lazy Class** — *what*: "a class that isn't doing enough to justify its maintenance." *Why it matters*: every class has a cost (to read, navigate, maintain); one that earns too little is pure overhead. *Fix*: Inline Class (fold it into its user), Collapse Hierarchy (if it's a thin subclass). 13. **Speculative Generality** — *what*: "classes and features have been added just because a need for them may arise someday." (YAGNI violation — abstraction with no current user.) *Why it matters*: unused flexibility is dead weight that confuses readers. *Fix*: Collapse Hierarchy, Inline Class, Remove Parameter, Rename Method. 14. **Temporary Field** — *what*: "an instance variable that is set only in certain circumstances" (otherwise empty/null). *Why it matters*: readers can't tell when the field is valid; it's a hidden conditional. *Fix*: Extract Class (to hold the field and its logic), Introduce Null Object. 15. **Message Chains** — *what*: "transitive visibility chains" (`a.getB().getC().getD()`). *Why it matters*: the client is coupled to the whole navigation structure; any intermediate change breaks it. *Fix*: Hide Delegate, Extract Method + Move Method.

Set 3 (Refactoring1 p.8): 16. **Middle Man** — *what*: "excessive delegation" — a class where most methods just forward to another class. *Why it matters*: the wrapper adds indirection without adding value. *Fix*: Remove Middle Man (let clients talk to the delegate directly). 17. **Inappropriate Intimacy** — *what*: "excessive interaction and coupling" between classes — they poke into each other's private parts. *Why it matters*: tight bidirectional coupling makes either class hard to change alone. *Fix*: Move Method/Field, Extract Class, Change Bidirectional Association to Unidirectional, Replace Inheritance with Delegation. 18. **Alternative Classes with Different Interfaces** — *what*: "classes that do the same thing but have different interfaces for what they do." *Why it matters*: clients can't treat them uniformly; you can't swap one for the other. *Fix*: Rename Method / Move Method to align the interfaces, or Unify Interfaces with Adapter (Kerievsky). 19. **Incomplete Library Class** — *what*: a library class is missing methods you need, but you can't edit its source. *Why it matters*: you're forced to work around it awkwardly. *Fix*: Introduce Foreign Method (one method) or Introduce Local Extension (several methods, via subclass/wrapper). 20. **Data Class** — *what*: "classes that have fields, getting and setting methods for the fields, and nothing else" (anaemic objects — data with no behaviour). *Why it matters*: behaviour that should live with the data is scattered into

other classes (often causing Feature Envy). *Fix*: Move Method (pull behaviour in), Encapsulate Field/Collection, Hide Methods. 21. **Refused Bequest** — *what*: "when subclasses do not fulfill the commitments of their superclasses" (inheriting but ignoring/overriding most of the parent → wrong inheritance). *Why it matters*: inheritance implies an "is-a" contract; refusing the bequest breaks Liskov substitution. *Fix*: Replace Inheritance with Delegation (the subclass holds the old parent as a field instead), Push Down Method/Field. 22. **Comments** — *what*: "when comments are used to compensate for bad code" — a comment explaining a tangled block. *Why it matters*: the comment is a deodorant masking a smell; the underlying code is still unclear. *Fix*: Extract Method + a self-documenting name so the code explains itself and the comment becomes unnecessary (the fix is **not** "delete all comments").

The 7 Refactoring Categories (Fowler)

What the categories are. Fowler groups the ~50 catalog refactorings into seven families by *the kind of structural problem each family solves* (Refactoring1 p.9). **What they're for:** the grouping is a navigation aid — once you've named the kind of mess you're in (badly packaged code? misplaced responsibilities? gnarly conditionals?), the category tells you which shelf of refactorings to browse. The seven families:

1. **Composing Methods** — *what*: refactorings that fix how code is "packaged" into methods (splitting, inlining, turning methods into objects). *Used for*: getting method-level structure right — short, well-named, single-purpose methods.
2. **Moving Features Between Objects** — *what*: "reassigning responsibilities" — moving methods and fields to the class where they belong. *Used for*: fixing Feature Envy, Large Class, Inappropriate Intimacy — putting behaviour next to the data it uses.
3. **Organizing Data** — *what*: "making data easier to work with" — encapsulating fields, replacing primitives/arrays with objects. *Used for*: fixing Primitive Obsession, Data Class, and exposed mutable state.
4. **Simplifying Conditional Expressions** — *what*: "making conditional logic less error-prone" — decomposing, consolidating, and replacing conditionals with polymorphism. *Used for*: taming complex `if / switch` logic (the G2 complexity target).
5. **Making Method Calls Simpler** — *what*: "making interfaces easy to understand and use" — renaming, reordering/removing parameters, separating queries from modifiers. *Used for*: fixing Long Parameter List and confusing/unsafe APIs.
6. **Dealing with Generalization** — *what*: "moving features around a hierarchy of inheritance" — pulling up, pushing down, extracting super/subclasses and interfaces. *Used for*: fixing duplication across siblings, Refused Bequest, and getting inheritance right.
7. **Big Refactorings** — *what*: "large-scale changes to code" that span many small refactorings and take a long time. *Used for*: deep structural problems (tangled hierarchies, procedural code, GUI/domain mixing) that no single small move can fix.

The Refactoring Catalog (every named refactoring on the slides)

Page numbers in parentheses are [Fowler99] book pages as printed on the slides. For each: *what the move does* and *when to reach for it* (which smell it answers).

Category 1 — Composing Methods (Refactoring1 p.11–13) - **Extract Method (110)** — *what*: "you have a code fragment that can be grouped together. Turn the fragment into a method whose name explains the purpose of the method." *When/why*: the single most common refactoring — reach for it whenever a block of code needs a comment to explain it, when a method is too long, or when the same fragment appears twice. The new method's name documents intent, and naming the fragment often reveals reuse and hidden bugs. *Fixes Long Method, Duplicated Code, Comments.* - **Inline Method (117)** — *what*: "a method's body is just as clear

as its name. Put the method's body into the body of its callers and remove the method." *When/why*: use it when a method's indirection earns nothing — the body is as obvious as the call — or to undo an over-eager extraction before re-grouping. (The inverse of Extract Method.) - **Replace Method with Method Object (135)** — *what*: "you have a long method that uses local variables in such a way that you cannot apply Extract Method. Turn the method into an object so that all the local variables become fields on that object. It can then be decomposed into other methods on the same object." *When/why*: the escape hatch for a long method whose tangled locals block plain Extract Method — promoting locals to fields lets you finally break the monster into well-named pieces. Listed as a G1 fix (BetterCode p.11).

Category 2 — Moving Features Between Objects (Refactoring1 p.15–22) - **Move Method (142)** — *what*: "a method is, or will be, using or used by more features of another class than the class on which it is defined. Create a new method with a similar body in the class it uses most; either turn the old method into a simple delegation, or remove it altogether." *When/why*: reach for it when a method talks more to another class than its own — it puts behaviour next to the data it uses. *Fixes Feature Envy* (and helps Shotgun Surgery, Inappropriate Intimacy). - **Move Field (146)** — *what*: "a field is, or will be, used by another class more than the class on which it is defined. Create a new field in the target class, and change all its users." *When/why*: the data-level counterpart of Move Method — relocate a field to the class that uses it most, usually as part of re-homing responsibilities (Extract Class, fixing Feature Envy). - **Extract Class (149)** — *what*: "you have one class doing work that should be done by two. Create a new class and move the relevant fields and methods from the old class into the new class." *When/why*: the primary cure for a class that has grown to hold two responsibilities — split it so each new class changes for one reason. *Fixes Large Class, Data Clumps, Divergent Change, Temporary Field*. - **Inline Class (154)** — *what*: "a class isn't doing very much. Move all its features into another class and delete it." *When/why*: use when a class no longer pulls its weight (often after other refactorings stripped it). *Fixes Lazy Class*. (Inverse of Extract Class.) - **Hide Delegate (157)** — *what*: "a client is calling a delegate class of an object. Create methods on the server to hide the delegate." *When/why*: reach for it when clients navigate through one object to reach another (`order.getCustomer().getManager()`); wrapping the hop in a server method decouples the client from the chain. *Fixes Message Chains / Inappropriate Intimacy*. - **Remove Middle Man (160)** — *what*: "a class is doing too much simple delegation. Get the client to call the delegate directly." *When/why*: the opposite situation — a class forwards so much that the indirection costs more than it hides. *Fixes Middle Man*. (Inverse of Hide Delegate — note the tension between these two; you pick based on which smell dominates: too much hidden delegation vs too much exposed chaining.) - **Introduce Foreign Method (162)** — *what*: "a server class you are using needs an additional method, but you can't modify the class. Create a method in the client class with an instance of the server class as its first argument." *When/why*: a quick fix when you need *one* missing method on a library/third-party class you can't edit. *Fixes Incomplete Library Class*. - **Introduce Local Extension (164)** — *what*: "a server class you are using needs several additional methods, but you can't modify the class. Create a new class that contains these extra methods. Make this extension class a subclass or a wrapper of the original." *When/why*: the heavier alternative to Introduce Foreign Method when you need *several* additions — bundle them into a proper subclass or wrapper. *Fixes Incomplete Library Class (when several methods are needed)*.

Category 3 — Organizing Data (Refactoring1 p.24–30) - **Self Encapsulate Field (171)** — *what*: "you are accessing a field directly, but the coupling to the field is becoming awkward. Create getting and setting methods for the field and use only those to access the field." *When/why*: use it when subclasses need to override how a field is computed, or when direct access is getting in the way of further refactoring — routing all access through accessors gives you a single place to add logic. - **Replace Data Value with Object (175)** — *what*: "you have a data item that needs additional data or behavior. Turn the data item into an object." *When/why*: the direct cure for Primitive Obsession — when a "simple" value (a string code, a number) starts needing validation or behaviour, give it a class. *Fixes Primitive Obsession*. - **Replace Array with Object**

(186) — *what*: "you have an array in which certain elements mean different things. Replace the array with an object that has a field for each element." *When/why*: when an array is being abused as a record (`row[0]` is the name, `row[1]` the age), turn it into an object with named fields so the meaning is explicit and type-checked. - **Duplicate Observed Data (189)** — *what*: "you have domain data available only in a GUI control, and domain methods need access. Copy the data to a domain object. Set up an observer to synchronize the two pieces of data." *When/why*: the move that begins separating domain logic from the UI — keep the data in a domain object and use the Observer pattern to keep the widget in sync. *Used to Separate Domain from Presentation; relevant to JHotDraw's MVC split.* - **Encapsulate Field (206)** — *what*: "there is a public field. Make it private and provide accessors." *When/why*: the basic encapsulation move — whenever a field is public, hide it behind getters/setters so the class controls access and can change representation later. - **Encapsulate Collection (208)** — *what*: "a method returns a collection. Make it return a read-only view and provide add/remove methods." *When/why*: use it whenever a getter hands out a live collection that callers could mutate behind the owner's back — return an unmodifiable view and force changes through controlled add/remove methods. *Protects internal collections from external mutation.* - **Replace Subclass with Fields (232)** — *what*: "you have subclasses that vary only in methods that return constant data. Change the methods to superclass fields and eliminate the subclasses." *When/why*: when subclasses differ only by constant return values (e.g. `Male` / `Female` differing only in a code), they're pulling their weight as data, not behaviour — collapse them into fields. *Fixes Lazy Class / over-use of inheritance.*

Category 4 — Simplifying Conditional Expressions (Refactoring1 p.32–36) - **Decompose Conditional (238)** — *what*: "you have a complicated conditional (if-then-else) statement. Extract methods from the condition, then part, and else parts." *When/why*: reach for it when a conditional's *intent* is buried in long boolean expressions and branch bodies — replacing each with a named method makes the logic read like prose. - **Consolidate Conditional Expression (240)** — *what*: "you have a sequence of conditional tests with the same result. Combine them into a single conditional expression and extract it." *When/why*: when several separate checks all lead to the same action, merge them into one named test so readers see "these are all the same case." - **Replace Nested Conditional with Guard Clauses (250)** — *what*: "a method has conditional behavior that does not make clear the normal path of execution. Use guard clauses for all the special cases." *When/why*: when deep `if/else` nesting hides the main flow, return early on each special/edge case so the "normal path" is the un-indented mainline. - **Replace Conditional with Polymorphism (255)** — *what*: "you have a conditional that chooses different behavior depending on the type of an object. Move each leg of the conditional to an overriding method in a subclass. Make the original method abstract." *When/why*: the canonical fix for Switch Statements that branch on a type — each subclass owns its case, so adding a new type means adding a class, not editing every switch (open/closed). *Also BetterCode Guideline 2's recommended refactoring (BetterCode p.18).* - **Introduce Null Object (260)** — *what*: "you have repeated checks for a null value. Replace the null value with a null object." *When/why*: when `if (x != null)` checks litter the code, replace `null` with an object that implements the same interface and does "nothing"/the neutral behaviour — the checks disappear. (Cross-listed in Kerievsky as a high-level pattern, HighLevelRefactoring p.12.)

Category 5 — Making Method Calls Simpler (Refactoring1 p.38–41) - **Separate Query from Modifier (279)** — *what*: "you have a method that returns a value but also changes the state of an object. Create two methods, one for the query and one for the modification." *When/why*: use it to enforce Command-Query Separation — a method that both returns data and has side effects is surprising and unsafe to call; split it so queries are side-effect-free and callable freely. - **Parameterize Method (283)** — *what*: "several methods do similar things but with different values contained in the method body. Create one method that uses a parameter for the different values." *When/why*: when you see near-identical methods that differ only by a hard-coded value (`tenPercentRaise`, `fivePercentRaise`), fold them into one method taking that value as a parameter. *Fixes a flavour of Duplicated Code.* - **Replace Parameter with Method (292)** — *what*:

"an object invokes a method, then passes the result as a parameter for a method. The receiver can also invoke this method. Remove the parameter and let the receiver invoke the method." *When/why*: when a parameter's value is something the receiver could compute itself, drop the parameter and let it fetch the value — shortening the call. *Shrinks parameter lists (Long Parameter List)*. - **Introduce Parameter Object (295)** — *what*: "you have a group of parameters that naturally go together. Replace them with an object." *When/why*: when the same cluster of parameters keeps travelling together, that cluster is a missing concept — wrap it in an object, which both shortens signatures and gives the new type a home for related behaviour. *Fixes Data Clumps and Long Parameter List; this is the named fix for BetterCode Guideline 4 "extract parameters into objects" (BetterCode p.25)*.

Category 6 — Dealing with Generalization (Refactoring1 p.43–50) - **Pull Up Constructor Body (325)** — *what*: "you have constructors on subclasses with mostly identical bodies. Create a superclass constructor; call this from the subclass methods." *When/why*: when subclass constructors duplicate setup, move the shared initialisation up to the superclass and call it via `super(...)` — removing duplication across the hierarchy. - **Extract Subclass (330)** — *what*: "a class has features that are used only in some instances. Create a subclass for that subset of features." *When/why*: when a class carries fields/methods used only by some of its instances (often signalled by a type code or a Temporary Field), split that variant out into a subclass. *Fixes Temporary Field / Large Class*. - **Extract Superclass (336)** — *what*: "you have two classes with similar features. Create a superclass and move the common features to the superclass." *When/why*: when two sibling classes share fields/methods, lift the commonality into a new shared parent. *Fixes Duplicated Code across siblings*. - **Extract Interface (341)** — *what*: "several clients use the same subset of a class's interface, or two classes have part of their interfaces in common. Extract the subset into an interface." *When/why*: when clients only depend on a slice of a class, or several classes share a usage contract, capture that contract as an interface so clients depend on the role, not the concrete class. - **Collapse Hierarchy (344)** — *what*: "a superclass and subclass are not very different. Merge them together." *When/why*: when a subclass has stopped justifying its own existence (often after other refactorings drained it), merge it back into its parent. *Fixes Lazy Class in a hierarchy*. - **Form Template Method (345)** — *what*: "you have two methods in subclasses that perform similar steps in the same order, yet the steps are different. Get the steps into methods with the same signature, so that the original methods become the same. Then you can pull them up." *When/why*: when sibling methods follow the same overall algorithm but vary in individual steps, factor the shared skeleton into a superclass template method and leave the varying steps as overridable hooks. *Implements the Template Method pattern; Kerievsky lists this as the fix for Duplicated Code (HighLevelRefactoring p.12)*. - **Replace Inheritance with Delegation (352)** — *what*: "a subclass uses only part of a superclass's interface or does not want to inherit data. Create a field for the superclass, adjust methods to delegate to the superclass, and remove the subclassing." *When/why*: the cure when inheritance was the wrong tool — the "subclass" isn't truly an "is-a" of its parent. Hold the former parent as a field and delegate, removing the brittle inheritance. *Fixes Refused Bequest / Inappropriate Intimacy via inheritance*. - **Replace Delegation with Inheritance (355)** — *what*: "you're using delegation and are often writing many simple delegations for the entire interface. Make the delegating class a subclass of the delegate." *When/why*: the opposite — when a class delegates *all* of another's interface and the "is-a" relationship genuinely holds, inheritance removes the boilerplate. (Inverse of the above.)

Category 7 — Big Refactorings (Refactoring1 p.52–58) — large, multi-step restructurings that play out over many small refactorings: - **Tease Apart Inheritance** — *what*: "you have an inheritance hierarchy that is doing two jobs at once. Create two hierarchies and use delegation to invoke one from the other." *When/why*: when one hierarchy is tangling two independent dimensions of variation (e.g. shape and rendering), split it into two and connect them by delegation. *Fixes Parallel Inheritance Hierarchies*. - **Convert Procedural Design to Objects** — *what*: "you have code written in a procedural style. Turn the data records into objects, break up the behavior, and move the behavior to the objects." *When/why*: when

migrating procedural/legacy code toward OO — give the data records behaviour and move logic onto them, replacing free functions with methods. - **Separate Domain from Presentation** — *what*: "you have GUI classes that contain domain logic. Separate the domain logic into separate domain classes." *When/why*: when business logic has leaked into UI classes, extract it into domain classes (the MVC split), so the domain can be tested and changed independently of the GUI. *Directly relevant to JHotDraw's GUI/domain layering and to MVC*. - **Extract Hierarchy** — *what*: "you have a class that is doing too much work, at least in part through many conditional statements. Create a hierarchy of classes in which each subclass represents a special case." *When/why*: when one class handles many variants via big conditionals, grow a subclass per case — the big-refactoring, whole-class cousin of Replace Conditional with Polymorphism. *Big-refactoring cousin of Replace Conditional with Polymorphism*.

The catalog slides' before/after figures — every worked example drawn in Refactoring1

Almost every catalog slide carries a concrete before→after example (code listing or UML mini-diagram) under the definition text. These are the *mechanics in miniature* — for the exam, being able to reproduce or recognise the example for a named refactoring is as valuable as reciting its definition. All of the following are read directly from the rendered slides.

Composing Methods examples:

- **Extract Method (110)** (Refactoring1 p.11) — the canonical `printOwing` example. Before: one method prints a banner and then prints details inline behind a `//print details` comment. After: the commented block becomes its own named method and the comment disappears.

```
// BEFORE
void printOwing() {
    printBanner();
    //print details
    System.out.println ("name:      " + _name);
    System.out.println ("amount    " + getOutstanding());
}

// AFTER
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}
void printDetails (double outstanding) {
    System.out.println ("name:      " + _name);
    System.out.println ("amount    " + outstanding);
}
```

Note how the *comment* in the before-version marks exactly where to extract, and the extracted method's *name* replaces the comment — this is the Comments-smell fix in action.

- **Inline Method (117)** (Refactoring1 p.12) — the `getRating` example. Before: `getRating()` returns `(moreThanFiveLateDeliveries()) ? 2 : 1`; where `moreThanFiveLateDeliveries()` merely returns `_numberOfLateDeliveries > 5`. After: the helper is deleted and `getRating()` returns `(_numberOfLateDeliveries > 5) ? 2 : 1`; directly. The slide shows when indirection earns nothing: the expression is as clear as the method name.
- **Replace Method with Method Object (135)** (Refactoring1 p.13) — the `Order.price()` example. Before: `price()` declares locals `double primaryBasePrice`; `double secondaryBasePrice`; `double tertiaryBasePrice`; `// long computation`; . After (UML): a new class `PriceCalculator` whose *fields* are

`primaryBasePrice`, `secondaryBasePrice`, `tertiaryBasePrice` and whose method is `compute`; `Order.price()` now just does `return new PriceCalculator(this).compute()`. The locals-become-fields move is exactly what unblocks further Extract Method inside `PriceCalculator`.

Moving Features Between Objects examples:

- **Move Method (142)** (Refactoring1 p.15) — generic UML: `Class 1` holds `aMethod()` before; after, `aMethod()` sits in `Class 2` and `Class 1` is empty of it.
- **Move Field (146)** (Refactoring1 p.16) — the mirror diagram for state: `aField` moves from `Class 1` to `Class 2`.
- **Extract Class (149)** (Refactoring1 p.17) — the `Person / Telephone Number` example. Before: `Person` has `name`, `officeAreaCode`, `officeNumber` and `getTelephoneNumber()`. After: `Person` keeps `name` and `getTelephoneNumber()` and holds an `officeTelephone` association (multiplicity 1) to a new class `Telephone Number` with `areaCode`, `number`, `getTelephoneNumber()`. The two phone fields were a hidden concept — the new class names it.
- **Inline Class (154)** (Refactoring1 p.18) — *the same Person / Telephone Number diagram run in reverse*: `Telephone Number` is folded back into `Person`, which ends up with `name`, `areaCode`, `number`, `getTelephoneNumber()`. The deck deliberately reuses one example for the inverse pair so you see they are two directions of one decision.
- **Hide Delegate (157)** (Refactoring1 p.19) — `Client Class` calls both `Person.getDepartment()` and `Department.getManager()` before; after, the client calls only `Person.getManager()` and `Person` privately delegates to `Department`. The client no longer knows `Department` exists.
- **Remove Middle Man (160)** (Refactoring1 p.20) — the same `Person / Department` trio reversed: the client talks to both `Person.getDepartment()` and `Department.getManager()` directly, removing `Person`'s forwarding method. Again an inverse pair drawn on one shared example.
- **Introduce Foreign Method / Introduce Local Extension** (Refactoring1 p.21) — *both* definitions share slide 21 (the slide titled "Introduce Foreign Method (162)" carries the Local Extension definition as its second half). Slide 22 then gives the Local Extension figure: before, a `Client Class` owns `nextDay(Date) : Date`; after, a subclass `MfDate` of `Date` owns `nextDay() : Date` — the missing methods now live on a proper extension type instead of being foreign methods scattered in clients (Refactoring1 p.22).

Organizing Data examples:

- **Self Encapsulate Field (171)** (Refactoring1 p.24) — the range-check example:

```
// BEFORE
private int _low, _high;
boolean includes (int arg) {
    return arg >= _low && arg <= _high;
}

// AFTER
private int _low, _high;
boolean includes (int arg) {
    return arg >= getLow() && arg <= getHigh();
}
int getLow() {return _low;}
int getHigh() {return _high;}
```

The point: once all access routes through `getLow()` / `getHigh()`, a subclass can override how the bounds are computed without touching `includes`.

- **Replace Data Value with Object (175)** (Refactoring1 p.25) — `Order` with `customer: String` becomes `Order` with a composition (filled diamond, multiplicity 1) to a `Customer` class holding `name: String`. The string was a customer all along.
- **Replace Array with Object (186)** (Refactoring1 p.26) — the football-performance example:

```
// BEFORE
String[] row = new String[3];
row [0] = "Liverpool";
row [1] = "15";

// AFTER
Performance row = new Performance();
row.setName("Liverpool");
row.setWins("15");
```

Index positions with secret meanings become named, typed properties.

- **Duplicate Observed Data (189)** (Refactoring1 p.27) — the `Interval Window` example. Before: a GUI class `Interval Window` owns `startField`, `endField`, `lengthField` (all `TextField`) and the logic `calculateLength`, `calculateEnd` alongside its `StartField_FocusLost` / `EndField_FocusLost` / `LengthField_FocusLost` handlers. After: the window keeps only the text fields and focus handlers and implements an «interface» `Observer`; a new domain class `Interval` holds `start`, `end`, `length` (as `String`) plus `calculateLength`, `calculateEnd` and extends `Observable`. Domain data is *duplicated* into the domain object and the Observer pattern keeps the two synchronized — the slide is a miniature MVC split.
- **Encapsulate Field (206)** (Refactoring1 p.28) — `public String _name` becomes `private String _name`; `public String getName()` {return `_name`;} `public void setName(String arg)` {`_name` = `arg`;}.
- **Encapsulate Collection (208)** (Refactoring1 p.29) — `Person` with `getCourses():Set` and `setCourses(:Set)` becomes `Person` with `getCourses():Unmodifiable Set`, `addCourse(:Course)`, `removeCourse(:Course)`. The whole-collection setter disappears; mutation is funneled through `add/remove`.
- **Replace Subclass with Fields (232)** (Refactoring1 p.30) — the `Person / Male / Female` example: subclasses `Male` and `Female` override `getCode()` to return `'M'` and return `'F'` respectively — constant data only. After: a single `Person` class with a `code` field and one `getCode()`; the subclasses are deleted.

Simplifying Conditional Expressions examples:

- **Decompose Conditional (238)** (Refactoring1 p.32) — the summer/winter charge example:

```
// BEFORE
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;

// AFTER
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
```

Condition, then-part and else-part each become a named method — the calendar logic now reads like the business rule it encodes.

- **Consolidate Conditional Expression (240)** (Refactoring1 p.33) — the `disabilityAmount` example: three sequential guards `if (_seniority < 2) return 0; if (_monthsDisabled > 12) return 0; if (_isPartTime) return 0;` collapse into a single `if (isNotEligibleForDisability()) return 0;`. Three checks, one *meaning* — the extraction names the meaning.
- **Replace Nested Conditional with Guard Clauses (250)** (Refactoring1 p.34) — the `getPayAmount` example. Before: nested `if (_isDead) ... else { if (_isSeparated) ... else { if (_isRetired) ... else ... } }` accumulating into a `result` variable returned at the end. After: flat guard clauses — `if (_isDead) return deadAmount(); if (_isSeparated) return separatedAmount(); if (_isRetired) return retiredAmount(); return normalPayAmount();`. Each special case exits immediately; the normal path is the unindented last line.
- **Replace Conditional with Polymorphism (255)** (Refactoring1 p.35) — the bird-speed example (Fowler's Monty Python birds):

```
// BEFORE
double getSpeed() {
    switch (_type) {
        case EUROPEAN:
            return getBaseSpeed();
        case AFRICAN:
            return getBaseSpeed() - getLoadFactor() * _numberOfCoconuts;
        case NORWEGIAN_BLUE:
            return (_isNailed) ? 0 : getBaseSpeed(_voltage);
    }
    throw new RuntimeException ("Should be unreachable");
}
```

After (UML): an abstract `Bird` with `getSpeed`, and subclasses `European`, `African`, `Norwegian Blue`, each overriding `getSpeed` with its own leg of the switch. Adding a new bird is a new subclass, not another `case` — and the unreachable-default boilerplate vanishes.

- **Introduce Null Object (260)** (Refactoring1 p.36) — the billing-plan example: `if (customer == null) plan = BillingPlan.basic(); else plan = customer.getPlan();` is replaced by a `Customer` superclass with `getPlan` and a `Null Customer` subclass whose `getPlan` returns the basic plan — the call site shrinks to just `plan = customer.getPlan();`.

Making Method Calls Simpler examples:

- **Separate Query from Modifier (279)** (Refactoring1 p.38) — `Customer.getTotalOutstandingAndSetReadyForSummaries` (one method, two jobs — the name itself confesses) splits into `getTotalOutstanding` (pure query) and `setReadyForSummaries` (pure modifier).
- **Parameterize Method (283)** (Refactoring1 p.39) — `Employee` with `fivePercentRaise()` and `tenPercentRaise()` becomes `Employee` with a single `raise(percentage)`. Two hard-coded twins fold into one parameterized method.
- **Replace Parameter with Method (292)** (Refactoring1 p.40) — the discount example:

```
// BEFORE
int basePrice = _quantity * _itemPrice;
discountLevel = getDiscountLevel();
double finalPrice = discountedPrice (basePrice, discountLevel);

// AFTER
int basePrice = _quantity * _itemPrice;
double finalPrice = discountedPrice (basePrice);
```

`discountedPrice` can call `getDiscountLevel()` itself, so the caller stops ferrying the value.

- **Introduce Parameter Object (295)** (Refactoring1 p.41) — `Customer` with `amountInvoicedIn(start: Date, end: Date)`, `amountReceivedIn(start: Date, end: Date)`, `amountOverdueIn(start: Date, end: Date)` becomes `amountInvoicedIn(DateRange)`, `amountReceivedIn(DateRange)`, `amountOverdueIn(DateRange)`. The recurring `(start, end)` pair was a missing `DateRange` concept — note this is also the textbook Data Clumps fix and the G4 fix.

Dealing with Generalization examples:

- **Pull Up Constructor Body (325)** (Refactoring1 p.43) — the `Manager` extends `Employee` example: the subclass constructor body `_name = name; _id = id; _grade = grade;` becomes `super(name, id); _grade = grade;` — the shared initialisation moves to the `Employee` constructor, the manager-specific line stays.
- **Extract Subclass / Extract Superclass (330/336)** (Refactoring1 p.44) — both definitions share one slide, emphasising they are the two directions of growing a hierarchy: push a partial feature set *down* into a new subclass vs pull shared features *up* into a new superclass.
- **Extract Interface (341)** (Refactoring1 p.45) — `Employee` with `getRate`, `hasSpecialSkill`, `getName`, `getDepartment`: clients that only bill don't need names/departments, so an «interface» `Billable` with just `getRate` and `hasSpecialSkill` is extracted and `Employee` implements it (dashed realization arrow).
- **Collapse Hierarchy (344)** (Refactoring1 p.46) — `Employee` ← `Salesman` where the subclass adds nothing distinct: merge into a single `Employee`.
- **Form Template Method (345)** (Refactoring1 p.47–48) — the billing-site example. Before: `Residential Site.getBillableAmount` computes `base = _units * _rate * 0.5; tax = base * Site.TAX_RATE * 0.2; return base + tax;` while `Lifeline Site.getBillableAmount` computes `base = _units * _rate; tax = base * Site.TAX_RATE; return base + tax;` — same *skeleton* (base, then tax, then sum), different *steps*. After: superclass `Site` owns concrete `getBillableAmount()` defined as `return getBaseAmount() + getTaxAmount();` with `getBaseAmount` / `getTaxAmount` abstract (italicised), and `Residential Site` / `LifelineSite` each override only those two hook methods. This is the Template Method pattern grown from duplication.
- **Replace Inheritance with Delegation (352)** (Refactoring1 p.49) — the classic `Stack` extends `Vector` example: a `Stack` is *not* a `Vector` (it refuses most of the bequest), so after the refactoring `Stack` holds a `Vector` field (association, multiplicity 1) and `Stack.isEmpty()` is implemented as `return _vector.isEmpty();`
- **Replace Delegation with Inheritance (355)** (Refactoring1 p.50) — the `Employee` / `Person` example: before, `Employee` holds a `Person` (multiplicity 1) and `Employee.getName()` is just `return person.getName();` — pure forwarding for the entire interface. After: `Employee` simply extends `Person` and the boilerplate disappears.

Big Refactorings examples:

- **Tear Apart Inheritance** (Refactoring1 p.52–53) — *note the slide title*: the deck calls it "Tear Apart Inheritance"; Fowler's book chapter is "Tease Apart Inheritance" — same refactoring. The `Deal` example: before, one hierarchy tangles two jobs — `Deal` ← `Active Deal` / `Passive Deal`, each of which then sprouts `Tabular Active Deal` / `Tabular Passive Deal` (deal kind × presentation style multiplying out, i.e. Parallel Inheritance / combinatorial growth). After: two hierarchies — `Deal` ← `Active Deal` / `Passive Deal`, plus a separate `Presentation Style` ← `Tabular Presentation Style` / `Single Presentation Style` — connected by a `Deal` → 1 `Presentation Style` association (delegation). Two dimensions of variation, two hierarchies.

- **Convert Procedural Design to Objects** (Refactoring1 p.54–55) — the order-calculator example: before, a procedural `Order Calculator` class holds `determinePrice(Order)` and `determineTaxes(Order)` while `Order` and `Order Line` are dumb records. After: `Order Calculator` is gone; `Order` and `Order Line` each own `getPrice()` and `getTaxes()` — the behaviour moved onto the data it operates on.
- **Separate Domain from Presentation** (Refactoring1 p.56) — the order-window example: a single `Order Window` (GUI + domain logic mixed) becomes `Order Window` → 1 `Order` — the GUI class keeps presentation, the new `Order` domain class takes the business logic.
- **Extract Hierarchy** (Refactoring1 p.57–58) — the billing-scheme example: one conditional-laden `Billing Scheme` class becomes a `Billing Scheme` superclass with subclasses `Business Billing Scheme`, `Residential Billing Scheme`, `Disability Billing Scheme` — each special case that used to be an `if`-branch is now a class.

Pattern to notice across the deck: the slides repeatedly draw *inverse pairs on the same example* (Extract Class/Inline Class on `Person – Telephone Number`; Hide Delegate/Remove Middle Man on `Client – Person – Department`; Replace Inheritance with Delegation / Replace Delegation with Inheritance on container/person examples). If an exam question shows you one of these diagrams reversed, name the inverse refactoring (Refactoring1 p.17–20, p.49–50).

The 10 SIG Maintainability Guidelines (BetterCode / "Building Maintainable Software")

What the SIG model is. BetterCode reframes refactoring as **measurable**: "Software Maintenance in 10 Guidelines... Measurable Software Metrics (bettercodehub.com)" (BetterCode p.3). **What it's for:** to remove subjectivity from maintainability. Each guideline has a **rule**, a **motivation**, a **level**, and a **measure** (a counting recipe + threshold), so a tool can score the codebase and tell you exactly what to refactor. **How it's applied:** measure → find units/modules/components over threshold → refactor → re-measure (the same loop as Fowler, but gated by numbers). The 10 guidelines and the level each lives at (BetterCode p.5 "Guideline Levels"):

#	Guideline	Level
1	Write Short Units of Code	Unit
2	Write Simple Units of Code	Unit
3	Write Code Once	Unit
4	Keep Unit Interfaces Small	Unit
5	Separate Concerns in Modules	Module
6	Couple Architecture Components Loosely	Component
7	Keep Architecture Components Balanced	Component
8	Keep Your Codebase Small	System
9	Automated Tests	System
10	Write Clean Code	Unit

What the four levels are. The SIG hierarchy (BetterCode p.9 "Guideline 1: Level") defines *the granularity at which each guideline is measured* — a guideline only makes sense at its level (you measure complexity per *unit*, coupling per *module*, balance per *component*, size per *system*):

- **Unit** — *what*: the smallest independently executable/testable block: *Method, Procedure, Function, subroutine*. Most guidelines (G1–G4, G10) live here because units are where readability and testability are decided.
- **Module** — *what*: a *File, Class, or Script* — a grouping of units. G5 (separating concerns) is measured here.
- **Component** — *what*: a top-

level grouping: *top-level directory, package, namespace, Maven sub-module, Visual Studio solution*. G6–G7 (architecture) live here. - **System** — *what*: the whole codebase, maintained by a team of developers. G8 (size) and G9 (tests) are system-wide concerns.

Guideline-by-guideline (each with *what the rule is, what it's for, the exact threshold/measure, and the named refactoring where given*):

- **G1 — Write Short Units of Code.** *Rule: "Limit the length of code units to 15 lines of code."* Either don't write units longer than 15 LOC, or split long units into smaller ones until each has at most 15 LOC. *What it's for*: short units are easy to **test**, **analyze**, and **reuse** — a 15-line method can be understood at a glance and tested in isolation, while a 100-line method hides logic and resists reuse (BetterCode p.7–8). *Measure*: "every line in the unit that is non-empty and does not contain only comments is a line of code" (BetterCode p.10). *Refactorings*: **Extract Method (110)**, **Replace Method with Method Object (135)** (BetterCode p.11). *This is the metric form of the Long Method smell.*
- **G2 — Write Simple Units of Code.** *Rule: "Limit the number of branch points per unit to 4"* (i.e. cyclomatic complexity ≤ 5 , since $CC = \text{branch points} + 1$). Achieve it by splitting complex units into simpler ones (BetterCode p.13). *What it's for*: simple units are easier to **modify** and **test** — each branch point doubles the paths through the code, and you need a test per path; fewer branches means fewer paths, fewer tests, fewer hiding places for bugs (BetterCode p.14). *Measure*: "every branch point (`if`, `case`, `for`, `&&`, `||`) is counted, and we add 1 to the total" → e.g. 2 branch points + 1 = cyclomatic complexity of 3 (BetterCode p.16). *Refactoring*: **Replace Conditional with Polymorphism (255)** (BetterCode p.18) — push each branch into a subclass so the unit itself becomes branch-free. *This is the metric form of complex conditional logic / Switch Statements.*
- **G3 — Write Code Once.** *Rule: "Do not copy code."* Write reusable/generic code or call existing methods instead of duplicating. *What it's for*: duplicated code is harder to analyze and to modify — "when code is copied, bugs need to be fixed at multiple places" (and one copy always gets missed) (BetterCode p.20–21). *Measure*: a **duplicated block is ≥ 6 lines of code** that appears more than once ("WRONG WAY GO BACK: Duplicated code blocks ≥ 6 lines of code") (BetterCode p.23). *Fix*: Extract Method / Pull Up Method / Extract Superclass. *This is the metric form of the Duplicated Code smell.*
- **G4 — Keep Unit Interfaces Small.** *Rule: "Limit the number of parameters per unit to at most 4."* Achieve it by **extracting parameters into objects**. *What it's for*: small interfaces are easier to understand, reuse, and modify — a method with seven parameters is hard to call correctly and signals that several arguments really form one concept (BetterCode p.26). *Measure*: count every parameter in the unit header (BetterCode p.25–28). *Named fix*: **Introduce Parameter Object (295)**. *This is the metric form of the Long Parameter List / Data Clumps smells.*
- **G5 — Separate Concerns in Modules.** *Rule: "Avoid large modules in order to achieve loose coupling between them"* — assign responsibilities to separate modules and hide implementation behind interfaces (BetterCode p.31). *What it's for*: loosely-coupled modules can be understood, changed, and tested in isolation; a change in one doesn't ripple into many. *Measure*: **module coupling = fan-in at file level** (every dependency from units *outside* module A into units *inside* module A increases A's coupling) plus module size (BetterCode p.34). *Threshold table* mapping fan-in → modifiability/testability: **1–10 Excellent, 11–20 Good, 21–50 Difficult, >50 Impossible** (BetterCode p.34) — a module everything depends on becomes effectively impossible to change safely.
- **G6 — Couple Architecture Components Loosely.** *Rule: "Achieve loose coupling between top-level components"* by minimizing the relative amount of code in modules exposed to (callable from) other components (BetterCode p.36). *What it's for*: when components hide most of their code behind a

thin interface, you can replace or test a whole component without touching the rest of the system.

Measure: count dependencies and cyclic dependencies among components; compute the **weighted percentage of hidden code** — *hidden* modules (no external calls / only calls to other modules) vs *interface* modules (incoming, or incoming+outgoing = "throughput"); the % hidden determines component independence (BetterCode p.39). *Example:* loosely-coupled components have few external dependencies and are easy to test; tightly-coupled ones are hard to test (BetterCode p.40).

- **G7 — Keep Architecture Components Balanced.** *Rule:* "**Balance the number and relative size of top-level components**" — organise so the number of components is **close to 9 (between 6 and 12)** and the components are of **approximately equal size** (BetterCode p.42). *What it's for:* a balanced architecture makes code easy to *find* (you can guess which component holds a feature) and *analyze*, and it isolates maintenance to one component (BetterCode p.43); too few huge components or too many tiny ones both defeat that. *Measure:* a combined calculation of (number of top-level components) and (the uniformity of component size) (BetterCode p.45).
- **G8 — Keep Your Codebase Small.** *Rule:* "**Keep your codebase as small as feasible**" — avoid codebase growth, actively reduce system size (BetterCode p.48). *What it's for:* size is the enemy of maintainability and of project success — "a project that sets out to build a large codebase is more likely to **fail**," and large systems have **higher defect density** (more bugs per line) (BetterCode p.49). Every line is a liability to read, test, and maintain, so the cheapest code is the code you don't write. *Measure:* **man-months = lines of code / average developer productivity (LOC per year)** — the number of person-months needed to (re)build the system, used as a size yardstick (BetterCode p.51).
- **G9 — Automated Tests.** *Rule:* "**Automate tests for your codebase**" using a test framework (BetterCode p.53). *What it's for:* automated tests make testing **repeatable**, development **efficient**, and code **predictable**; **tests document the code** (a test shows how a unit is meant to be used); and **writing tests makes you write better code** (testable code is decoupled code) (BetterCode p.54). *Why it's central here:* this is the safety net that makes refactoring's behaviour-preservation *verifiable* rather than hoped-for — you refactor, re-run the suite, and trust green tests. Lives at System level (BetterCode p.55). Links to the Verification phase.
- **G10 — Write Clean Code.** *Rule:* "**Write clean code**" — **don't leave code smells behind after development work** ("clean code is maintainable code") (BetterCode p.57). *What it's for:* it operationalises the Boy-Scout rule — small messes left behind compound into unmaintainable code, so each piece of work must finish clean. *How (the seven "How To" rules, BetterCode p.58):* leave behind (1) no **unit-level code smells**, (2) no **bad comments**, (3) no **code in comments** (commented-out code), (4) no **dead code**, (5) no **long identifier names**, (6) no **magic constants** (unexplained literals), (7) no **badly handled exceptions**. *This is literally the Postfactoring discipline expressed as a checklist.*

The SIG "Concepts" table — Unit, Module, Component, System mapped to Java (BetterCode p.4)

Before the guidelines, the deck pins down its four measurement granularities in a three-column table — *generic name*, *generic definition*, and what each maps to in *Java* (BetterCode p.4). The Java column is exam bait: note that the two architecture-level concepts have **no language construct at all**.

Generic name	Generic definition	In Java
Unit	Smallest grouping of lines that can be executed independently	Method or constructor
Module	Smallest grouping of units	Top-level class, interface, or enum
Component	Top-level division of a system as defined by its software architecture	(Not defined by the language)

Generic name	Generic definition	In Java
System	The entire codebase under study	(Not defined by the language)

Three things to memorise from this table: (1) a *constructor* counts as a unit just like a method; (2) a module is a **top-level** class/interface/enum — nested classes don't start a new module; (3) component and system exist only in the *architecture*, not in Java syntax — which is why G6/G7 tooling needs you to tell it what the components are (top-level directories, packages, Maven sub-modules, Visual Studio solutions — the examples given on the Level slides, BetterCode p.9).

The exact Rule–How–Why slide for each guideline (BetterCode rendered pages)

Every guideline's opening slide has the same three-bullet anatomy: the **rule** (bold imperative), the **how** ("Do this by ..."), and the **why** ("This improves maintainability because ..."). Reproducing the precise wording per guideline — useful because exam questions often quote the "because" clause and ask which guideline it belongs to:

- **G1 (BetterCode p.7): "Limit the length of code units to 15 lines of code.** Do this by **not writing units that are longer than 15 lines of code** in the first place, or by **splitting long units into multiple smaller units** until each unit has at most 15 lines of code. This improves maintainability because **small units are easy to understand, easy to test, and easy to reuse.**"
- **G2 (BetterCode p.13): "Limit the number of branch points per unit to 4.** Do this by **splitting complex units into simpler ones** and avoiding complex units altogether. This improves maintainability because keeping the number of branch points low **makes units easier to modify and test.**"
- **G3 (BetterCode p.20): "Do not copy code.** Do this by **writing reusable, generic code and/or calling existing methods instead.** This improves maintainability because **when code is copied, bugs need to be fixed at multiple places**, which is inefficient and error-prone."
- **G4 (BetterCode p.25): "Limit the number of parameters per unit to at most 4.** Do this by **extracting parameters into objects.** This improves maintainability because keeping the number of parameters low **makes units easier to understand and reuse.**"
- **G5 (BetterCode p.31): "Avoid large modules in order to achieve loose coupling between them.** Do this by **assigning responsibilities to separate modules and hiding implementation details behind interfaces.** This improves maintainability because **changes in a loosely coupled codebase are much easier to oversee and execute** than changes in a tightly coupled codebase."
- **G6 (BetterCode p.36): "Achieve loose coupling between top-level components.** Do this by **minimizing the relative amount of code within modules that is exposed to (i.e., can receive calls from) modules in other components.** This improves maintainability because **independent components ease isolated maintenance.**"
- **G7 (BetterCode p.42): "Balance the number and relative size of top-level components in your code.** Do this by **organizing source code in a way that the number of components is close to 9 (i.e., between 6 and 12)** and that the **components are of approximately equal size.** This improves maintainability because **balanced components ease locating code and allow for isolated maintenance.**"
- **G8 (BetterCode p.48): "Keep your codebase as small as feasible.** Do this by **avoiding codebase growth and actively reducing system size.** This improves maintainability because having a **small product, project, and team is a success factor.**"
- **G9 (BetterCode p.53): "Automate tests for your codebase.** Do this by **writing automated tests using a test framework.** This improves maintainability because **automated testing makes**

development predictable and less risky."

- **G10 (BetterCode p.57): "Write clean code.** Do this by **not leaving code smells behind after development work.** This improves maintainability because **clean code is maintainable code."**

The full motivation slides add the named sub-reasons: G1 — short units are easy to *test*, easy to *analyze*, easy to *reuse* (BetterCode p.8); G2 — simple units are easier to *modify* and easier to *test* (BetterCode p.14); G3 — duplicated code is harder to *analyze* and harder to *modify* (BetterCode p.21); G4 — small interfaces are easier to *understand and reuse*, and methods with small interfaces are easier to *modify* (BetterCode p.26); G5 — small, loosely coupled modules (a) allow developers to work on isolated parts of the codebase, (b) ease navigation through the codebase, and (c) prevent no-go areas for new developers (BetterCode p.32); G6 — low component dependence (a) allows for isolated maintenance, (b) separates maintenance responsibilities, and (c) eases testing (BetterCode p.37); G7 — a good component balance (a) eases finding and analyzing code, (b) better isolates maintenance effects, and (c) separates maintenance responsibilities (BetterCode p.43); G8 — (a) a project that sets out to build a large codebase is more likely to fail, and (b) large systems have higher defect density (BetterCode p.49); G9 — automated testing (a) makes testing repeatable, (b) makes development efficient, (c) makes code predictable, (d) tests document the code that is tested, and (e) writing tests makes you write better code (BetterCode p.54).

The Guideline Levels overview slide — wording and ordering notes (BetterCode p.5, 9)

The overview slide (BetterCode p.5) lists the ten guidelines with a line from each to a nested level diagram (Unit inside Module inside Component inside System). Two trip-wires for anyone who memorised only the chapter titles:

1. **Shortened titles.** On p.5 the guidelines are worded "1. Keep Units Short — 2. Keep Units Simple — 3. Keep Interfaces Small — 4. Write Code Once — 5. Separate Concerns in Modules — 6. Couple Architecture Components Loosely — 7. Keep Architecture Components Balanced — 8. Keep Your Codebase Small — 9. Automate Tests — 10. Write Clean Code." So "Keep Units Short" = "Write Short Units of Code", "Automate Tests" = "Automated Tests", etc.
2. **Items 3 and 4 are swapped** relative to the chapter order: on the p.5 overview, item 3 is *Keep Interfaces Small* and item 4 is *Write Code Once*, whereas the guideline chapters number them G3 = Write Code Once (BetterCode p.19–23) and G4 = Keep Unit Interfaces Small (BetterCode p.24–29). If a question references "guideline 3", check which numbering it uses.

The per-guideline "Level" slides (BetterCode p.9, 15, 22, 33, 55 and siblings) all reuse one annotated diagram: a System box ("4. System — codebase maintained by a team of developers") containing Components ("3. Component — e.g.: top-level directory, package, namespace, Maven sub-module, Visual Studio solution"), containing Modules ("2. Module — e.g.: File, Class, Script"), containing Units ("1. Unit — e.g.: Method, Procedure, Function, subroutine"), with the level relevant to the current guideline highlighted.

Guideline 10's seven "leave no ... behind" rules, one by one (BetterCode p.58)

The G10 "How To" slide is a seven-item checklist, each phrased "Leave no X behind." Verbatim list, with what each item targets:

1. **"Leave no unit-level code smells behind."** — the catch-all: before you finish, the units you touched must pass the unit-level guidelines (G1 length, G2 complexity, G3 duplication, G4 interface size) and be free of Fowler's method-level smells [**course-context inference for the G1–G4 linkage; the rule text is verbatim**].

2. **"Leave no bad comments behind."** — comments that mislead, restate the obvious, or compensate for unclear code; the direct SIG echo of Fowler's *Comments* smell, whose fix is Extract Method + a self-documenting name (Refactoring1 p.8).
3. **"Leave no code in comments behind."** — commented-out code blocks. Distinct from rule 2: this is *code* hiding in comments (kept "just in case"), which confuses readers and rots instantly; version control makes keeping it pointless.
4. **"Leave no dead code behind."** — unreachable or never-called code; the executable sibling of rule 3 and a cousin of *Speculative Generality* (Refactoring1 p.7).
5. **"Leave no long identifier names behind."** — names so long they hurt readability; note the direction — SIG flags *long* identifiers, not short ones, as the smell here.
6. **"Leave no magic constants behind."** — unexplained literal values (`0.5` , `9` , `"multipart/form-data"`) embedded in logic with no named constant to carry their meaning.
7. **"Leave no badly handled exceptions behind."** — swallowed, blanket-caught, or ignored exceptions; error handling left sloppy after a change.

Why this list matters for the course: it is the *operational definition of Postfactoring* — a literal exit checklist you run before Conclusion, and the reason the guide equates G10 with the Boy-Scout rule (BetterCode p.57–58) [course-context inference for the process placement].

High-Level / Composite Refactoring (Kerievsky — "Refactoring to Patterns")

What this layer is. The third deck adds a layer *above* Fowler: refactoring not just to clean code but **toward design patterns** (HighLevelRefactoring p.2). **What it's for:** Fowler's moves fix local messes; Kerievsky's composite moves fix *design* problems by deliberately growing a known pattern out of smelly code — and, crucially, teach the *thinking* that lets you do this for problems no book lists. Core ideas:

- **"Design patterns are the word problems of the programming world; refactoring is its algebra."** *What it means:* after reading *Design Patterns* [GHJV94] you often think "if I'd only known this pattern my system would be cleaner today" (HighLevelRefactoring p.8–9). Rather than trying to design the pattern in up front, you *reach* it by applying a sequence of small behaviour-preserving moves — solving the design "word problem" with refactoring "algebra".
- **The algebra of refactoring:** *what/why:* the value is not memorising the steps of any one composite refactoring but learning the **thought process** — "you learn to solve design problems in behavior-preserving steps, and you are not bound by the small subset of actual problems" the book lists (HighLevelRefactoring p.10). The skill generalises beyond the catalog.
- **High-level vs low-level refactoring:** *the distinction* — Fowler's are *low-level* (Extract Method, Move Field — small, mechanical, single-step, often tool-automatable). Kerievsky's are *high-level / composite* (Replace Conditional Logic with Strategy, Move Embellishment to Decorator) — each is a *sequence* of low-level refactorings that **introduces a design pattern**. *Why it matters:* high-level moves give you a target (a named pattern) and a plan; low-level moves are the steps that get you there. [course-context inference: "high-level" = composite/pattern-directed; "low-level" = atomic catalog moves.]
- **Benefits of composite refactorings** (HighLevelRefactoring p.16) — *what they give you:* an **overall plan** for a refactoring sequence (you know where you're heading); they **suggest non-obvious design directions** (a pattern you might not have considered); and they give **insights into implementing patterns** (you learn the pattern by growing it, not by copying a diagram).
- **A pattern can be reached many ways / from different directions.** *What/why:* there's no single canonical implementation of a pattern, and you can refactor **towards** a pattern, further **toward/into** it, or **away** from it. Over-applying a pattern is itself a smell, so Kerievsky's "Refactoring Directions" table

deliberately lists moves *away* from patterns too (e.g. *Inline Singleton (114)* moves away from Singleton when a singleton is no longer warranted) (HighLevelRefactoring p.21–22).

- **Two refactoring heuristics** (HighLevelRefactoring p.20): **Automation First** — *what*: "manual refactorings are dirt roads, automated refactorings are highways; look first for the highways." *Why*: tool-driven moves are faster and safer (the IDE preserves behaviour mechanically), so prefer them. **Client First** — *what*: to find a simpler automated path, start refactoring from a *client* of the smelly code. *Why*: changing how callers use the code often reveals an easier, tool-supported route into the target structure.

Kerievsky's Code-Smell → Refactoring catalog (HighLevelRefactoring p.12–13). *What this table is for*: it maps each smell to the pattern-introducing refactoring(s) that resolve it — the high-level analog of Fowler's smell→refactoring pairing. [F] marks a smell also in Fowler:

Code Smell	Refactoring(s) toward a pattern
Alternative Classes w/ Different Interfaces (43) [F]	Unify Interfaces with Adapter (247)
Combinatorial Explosion (45)	Replace Implicit Language with Interpreter (269)
Conditional Complexity (41)	Replace Conditional Logic with Strategy (129); Move Embellishment to Decorator (144); Replace State-Altering Conditionals with State (166); Introduce Null Object (301)
Duplicated Code (39) [F]	Form Template Method (205); Introduce Polymorphic Creation with Factory Method (88); Chain Constructors (340); Replace One/Many Distinctions with Composite (224); Extract Composite (214); Unify Interfaces with Adapter (247); Introduce Null Object (301)
Indecent Exposure (42)	Encapsulate Classes with Factory (80)
Large Class (44) [F]	Replace Conditional Dispatcher with Command (191); Replace State-Altering Conditionals with State (166); Replace Implicit Language with Interpreter (269)
Lazy Class (43) [F]	Inline Singleton (114)
Long Method (40) [F]	Compose Method (123); Move Accumulation to Collecting Parameter (313); Replace Conditional Dispatcher with Command (191); Move Accumulation to Visitor (320); Replace Conditional Logic with Strategy (129)
Oddball Solution (45)	Unify Interfaces with Adapter (247)
Primitive Obsession (41) [F]	Replace Type Code with Class (286); Replace State-Altering Conditionals with State (166); Replace Conditional Logic with Strategy (129); Replace Implicit Tree with Composite (178); Replace Implicit Language with Interpreter (269); Move Embellishment to Decorator (144); Encapsulate Composite with Builder (96)
Solution Sprawl (43)	Move Creation Knowledge to Factory (68)
Switch Statements (44) [F]	Replace Conditional Dispatcher with Command (191); Move Accumulation to Visitor (320)

The deck also gives a **study sequence** of 21 sessions ordering these refactorings for learning (HighLevelRefactoring p.14–15) — *what for*: a pedagogical path through the catalog so each refactoring builds on earlier ones. And a **Refactoring Directions** matrix mapping each GoF pattern (Adapter, Builder, Collecting Parameter, Command, Composite, Creation Method, Decorator, Factory, Factory Method,

Interpreter, Iterator, Null Object, Observer, Singleton, State, Strategy, Template Method, Visitor) to the moves that go *towards* and *away* from it (HighLevelRefactoring p.21–22) — *what for*: it lets you navigate in either direction, reinforcing that *adding* a pattern and *removing* an over-applied one are both legitimate refactorings.

What is a pattern? — Alexander's three-part rule (HighLevelRefactoring p.7)

The deck grounds "pattern" in the original source — Christopher Alexander (the architect whose work inspired the GoF), cited as [Alexander, ATWoB, p247] (HighLevelRefactoring p.7):

"Each pattern is a **three-part rule, which expresses a relation between a certain context, a problem, and a solution**. As an element in the world, each pattern is a relationship between a certain context, a certain system of forces which occurs repeatedly in that context, and a certain spatial configuration which allows these forces to resolve themselves. As an element of language, a pattern is an instruction, which shows how this spatial configuration can be used, over and over again, to resolve the given system of forces, wherever the context makes it relevant. The pattern is, in short, at the same time a **thing**, which happens in the world, and the **rule** which tells us how to create that thing, and when we must create it. It is both a **process and a thing**; both a description of a thing which is alive, and a description of the process which will generate that thing."

The three parts to remember: **context + problem + solution**. And the dual nature: a pattern is *simultaneously* the artifact (the thing) and the recipe (the process/rule for creating it) — which is exactly why Kerievsky can treat a pattern as a refactoring *destination*: the "process" half of the pattern is what a composite refactoring walks through (HighLevelRefactoring p.7).

The philosophy slides verbatim — Jefferson/Franklin, algebra, and the primer (HighLevelRefactoring p.6, 8–11)

Four short slides carry the deck's entire philosophy; quoting them exactly pays off because exam questions tend to quote *fragments* of them and ask what the fragment means.

Revision of The Declaration of Independence (p.6). The slide juxtaposes two drafts: "In 1776, Thomas Jefferson wrote the following un-famous words: '**We hold these truths to be sacred and undeniable...**'" and "In 1776, Benjamin Franklin revised Jefferson's words to read: '**We hold these truths to be self-evident...**'" — note the slide's framing that Jefferson's original is the *un-famous* version. The lesson: the *meaning* of the sentence survived while its *expression* improved — Franklin's edit is behaviour-preserving refactoring applied to prose. Iterative revision of expression, with meaning held constant, is the entire idea of refactoring transplanted outside code (HighLevelRefactoring p.6).

Algebra and Word Problems (p.8). Verbatim: "In algebra class, we first learn different manipulations, like: '*add the same value to both sides of the equation*', '*the commutative property of addition allows us to swap its operands*.' Once we know the manipulations, we're given word problems: '*A train leaves New York heading West...*' To solve this problem, you express it in terms of an algebraic equation and then apply the rules of algebra to arrive at an answer." The mapping the next slide will make: manipulations = individual refactorings (small, rule-like, always-valid moves); word problems = design problems; expressing the problem and applying rules = refactoring your way to a pattern (HighLevelRefactoring p.8).

Refactoring and Patterns (p.9). Verbatim: "**Design patterns are the word problems of the programming world; refactoring is its algebra.** After having read *Design Patterns* [DP], you reach a point where you say to yourself, '*If I had only known this pattern, my system would be so much cleaner*'

today.' The book you are holding introduces you to several sample problems, with solutions expressed in the operations of refactoring." Two things to retain: the regret quote (knowing patterns *after* the fact) is the motivating emotion, and the cure offered is not "design with patterns up front" but "*solutions expressed in the operations of refactoring*" — reach the pattern by moves, don't guess it in advance (HighLevelRefactoring p.9).

The Algebra of Refactoring (p.10). Verbatim: "Many people will read this book and try to **memorize the steps** to implement these patterns. Others will read this book and **clamor for these larger refactorings to be added to existing programming tools. Both of these approaches are misguided.** The true value of this book lies not in the actual steps to achieve a particular pattern but in understanding the **thought processes** that lead to those steps. By learning to think in the algebra of refactoring, you learn to solve design problems in behavior-preserving steps, and you are not bound by the small subset of actual problems that this book represents." Note that the slide rejects *two* named wrong approaches — rote memorisation *and* demanding tool automation of the composite refactorings — before stating the right one (internalise the thought process) (HighLevelRefactoring p.10).

Patterns of Refactoring (p.11). Verbatim: "So take these exemplars that Josh has laid out for you. **Study them. Find the underlying patterns of refactoring that are occurring. Seek the insights that led to the particular steps. Don't use this as a reference book, but as a primer.**" The instruction "primer, not reference book" is the study advice for the whole high-level catalog — and, by extension, for the lab portfolio, which asks you to explain *reasoning and strategy*, not just to name the refactoring you applied (HighLevelRefactoring p.11; RefactLab p.1).

The "Patterns Happy?" Hello World — the full code (HighLevelRefactoring p.3–5)

The deck opens with three slides titled "Patterns Happy?" showing one complete program. Read it before reading any commentary — the punchline is what the program actually does:

```
// (HighLevelRefactoring p.3)
interface MessageStrategy {
    public void sendMessage();
}

abstract class AbstractStrategyFactory {
    public abstract MessageStrategy createStrategy(MessageBody mb);
}

class MessageBody {
    Object payload;
    public Object getPayload() {
        return payload;
    }
    public void configure(Object obj) {
        payload = obj;
    }
    public void send(MessageStrategy ms) {
        ms.sendMessage();
    }
}
```

```
// (HighLevelRefactoring p.4)
class DefaultFactory extends AbstractStrategyFactory {
    private DefaultFactory() {
    }
    static DefaultFactory instance;
    public static AbstractStrategyFactory getInstance() {
```



```

        if (instance == null)
            instance = new DefaultFactory();
        return instance;
    }

    public MessageStrategy createStrategy(final MessageBody mb) {
        return new MessageStrategy() {
            MessageBody body = mb;
            public void sendMessage() {
                Object obj = body.getPayload();
                System.out.println(obj);
            }
        };
    }
}

```

```

// (HighLevelRefactoring p.5)
public class HelloWorld {
    public static void main(String[] args) {
        MessageBody mb = new MessageBody();
        mb.configure("Hello World!");
        AbstractStrategyFactory asf = DefaultFactory.getInstance();
        MessageStrategy strategy = asf.createStrategy(mb);
        mb.send(strategy);
    }
}

```

This entire three-slide program prints `Hello World!`. Patterns you can identify in it: **Strategy** (the `MessageStrategy` interface whose `sendMessage` is the pluggable behaviour), **Factory Method / Abstract Factory machinery** (`AbstractStrategyFactory.createStrategy` overridden by `DefaultFactory`), **Singleton** (`DefaultFactory`'s private constructor + static `instance` + lazy `getInstance()`), and an **anonymous inner class** implementing the strategy. The slide title — "Patterns Happy?" — is the question the lecture wants you to ask: is this *good* design, or is it pattern-drunk over-engineering for a one-line job? It is the opening exhibit for why the deck later insists you can refactor **away** from patterns, not just toward them (HighLevelRefactoring p.3–5, 21–22) **[course-context inference for the interpretation; the code and the title are verbatim from the slides]**. The Fowler-vocabulary diagnosis of this program would be Speculative Generality (Refactoring1 p.7).

Pattern: Factory Method — structure, and why structure diagrams are only examples (HighLevelRefactoring p.17–19)

The deck includes one full pattern write-up as its exemplar. The **structure** (HighLevelRefactoring p.17): an abstract `Creator` declares `factoryMethod() : Product` (italic = abstract) and a concrete `anOperation() : void` whose body uses `product = factoryMethod();`; a `ConcreteCreator` subclass overrides `factoryMethod() : Product` with `return new ConcreteProduct(...)`; on the product side, `Product` is the interface and `ConcreteProduct` implements it, with dashed dependency arrows from the creators to the products. The essence: *the superclass owns the algorithm, the subclass owns the choice of which concrete product to instantiate*.

Slide 18 then draws **the same pattern twice with different structures** ("There Are Many Ways To Implement A Pattern!"): in one variant `Creator.factoryMethod()` is concrete and itself returns `new ConcreteProduct(...)` (a default implementation the subclass may override); in the other, `factoryMethod()` is abstract in `Creator` and the `ConcreteCreator` returns `new Product(...)` (HighLevelRefactoring p.18). Slide 19 backs this with the Vlissides quote (cited [Vlissides, C++ Report, April 1998]):

"It seems you can't overemphasize that a **pattern's Structure diagram is just an example, not a specification**. It portrays the implementation we see most often. As such the Structure diagram will probably have a lot in common with your own implementation, but differences are inevitable and actually desirable. At the very least you will rename the participants as appropriate for your domain. Vary the implementation trade-offs, and your implementation might start looking a lot different from the Structure diagram." (HighLevelRefactoring p.19)

Exam-relevant consequence: "implementation X doesn't match the GoF diagram, therefore it isn't pattern Y" is a **false** statement — the diagram portrays the *most common* implementation, and deviations are "inevitable and actually desirable" (HighLevelRefactoring p.18–19).

The complete Study Sequence — all 21 sessions (HighLevelRefactoring p.14–15)

Kerievsky's recommended learning order through the composite-refactoring catalog, reproduced in full. Numbers in parentheses are [Ker05] book pages. Notice the pedagogy: creational refactorings first (sessions 1–3), the two workhorses Strategy and Template Method next (4–5), then method composition, the Composite cluster, behavioural patterns, and the hardest (Visitor, Interpreter) last:

Session	Refactoring(s)
1	Replace Constructors with Creation Methods (57); Chain Constructors (340)
2	Encapsulate Classes with Factory (80)
3	Introduce Polymorphic Creation with Factory Method (88)
4	Replace Conditional Logic with Strategy (129)
5	Form Template Method (205)
6	Compose Method (123)
7	Replace Implicit Tree with Composite (178)
8	Encapsulate Composite with Builder (96)
9	Move Accumulation to Collecting Parameter (313)
10	Extract Composite (214); Replace One/Many Distinctions with Composite (224)
11	Replace Conditional Dispatcher with Command (191)
12	Extract Adapter (258); Unify Interfaces with Adapter (247)
13	Replace Type Code with Class (286)
14	Replace State-Altering Conditionals with State (166)
15	Introduce Null Object (301)
16	Inline Singleton (114); Limit Instantiation with Singleton (296)
17	Replace Hard-Coded Notifications with Observer (236)
18	Move Embellishment to Decorator (144); Unify Interfaces (343); Extract Parameter (346)
19	Move Creation Knowledge to Factory (68)
20	Move Accumulation to Visitor (320)
21	Replace Implicit Language with Interpreter (269)

Three refactorings appear *only* in the study sequence and not in the smell table: **Replace Constructors with Creation Methods (57)**, **Extract Adapter (258)**, **Limit Instantiation with Singleton (296)**, plus the helper moves **Unify Interfaces (343)** and **Extract Parameter (346)** in session 18 (HighLevelRefactoring p.14–15). Session 16 deliberately pairs the *towards*-Singleton move (Limit Instantiation with Singleton) with the *away*-from-Singleton move (Inline Singleton) — direction is part of the curriculum.

The complete Refactoring Directions matrix (HighLevelRefactoring p.21–22)

The closing table has **four columns** — **Pattern** | **To** | **Towards** | **Away** (not two): "To" = the composite refactoring that takes you *to* (fully into) the pattern; "Towards" = a move that brings code *toward* the pattern without necessarily completing it; "Away" = the move that removes/re retreats from the pattern. Reproduced in full ("—" = the slide leaves that cell empty):

Pattern	To	Towards	Away
Adapter	Extract Adapter (258), Unify Interfaces with Adapter (247)	Unify Interfaces with Adapter (247)	—
Builder	Encapsulate Composite with Builder (96)	—	—
Collecting Parameter	Move Accumulation to Collecting Parameter (313)	—	—
Command	Replace Conditional Dispatcher with Command (191)	Replace Conditional Dispatcher with Command (191)	—
Composed Method	Compose Method (123)	—	—
Composite	Replace One/Many Distinctions with Composite (224), Extract Composite (214), Replace Implicit Tree with Composite (178)	—	Encapsulate Composite with Builder (96)
Creation Method	Replace Constructors with Creation Methods (57)	—	—
Decorator	Move Embellishment to Decorator (144)	Move Embellishment to Decorator (144)	—
Factory	Move Creation Knowledge to Factory (68), Encapsulate Classes with Factory (80)	—	—
Factory Method	Introduce Polymorphic Creation with Factory Method (88)	—	—
Interpreter	Replace Implicit Language with Interpreter (269)	—	—
Iterator	—	—	Move Accumulation to Visitor (320)
Null Object	Introduce Null Object (301)	—	—
Observer	Replace Hard-Coded Notifications with Observer (236)	Replace Hard-Coded Notifications with Observer (236)	—
Singleton	Limit Instantiation with Singleton (296)	—	Inline Singleton (114)

Pattern	To	Towards	Away
State	Replace State-Altering Conditionals with State (166)	Replace State-Altering Conditionals with State (166)	—
Strategy	Replace Conditional Logic with Strategy (129)	Replace Conditional Logic with Strategy (129)	—
Template Method	Form Template Method (205)	—	—
Visitor	Move Accumulation to Visitor (320)	Move Accumulation to Visitor (320)	—

Readings worth highlighting for the exam: (1) **Singleton** is the showcase row — a named move in *both* the To column (Limit Instantiation with Singleton) and the Away column (Inline Singleton); (2) the **Iterator** row has *only* an Away entry — Move Accumulation to Visitor takes you away from Iterator (because the accumulation logic stops iterating externally and becomes a Visitor); (3) the **Composite/Builder** interplay — Encapsulate Composite with Builder is *To* Builder but *Away* from Composite (wrapping the composite hides it); (4) several rows (Command, Decorator, Observer, State, Strategy, Visitor) list the *same* refactoring under both To and Towards — the move can either complete the pattern or just make progress toward it depending on how far you take it (HighLevelRefactoring p.21–22).

The five Kerievsky-only smells (HighLevelRefactoring p.12–13)

Kerievsky's smell→refactoring table mixes Fowler smells (marked [F]) with five smells of his own. The slides give names, [Ker05] page numbers and the paired refactorings; for the exam, know which five are *not* in Fowler's 22 and what each one's fix implies about its meaning:

- **Combinatorial Explosion (45)** — paired with *Replace Implicit Language with Interpreter (269)*: many code variants multiplying out to express what is really a small implicit language of rules/queries.
- **Conditional Complexity (41)** — paired with *Replace Conditional Logic with Strategy (129)*, *Move Embellishment to Decorator (144)*, *Replace State-Altering Conditionals with State (166)*, *Introduce Null Object (301)*: Kerievsky's broader umbrella over complicated conditional logic — note all four fixes are behavioural patterns.
- **Indecent Exposure (42)** — paired with *Encapsulate Classes with Factory (80)*: classes/constructors visible to clients that shouldn't see them; the factory hides them.
- **Oddball Solution (45)** — paired with *Unify Interfaces with Adapter (247)*: the same problem solved one way almost everywhere but differently in one spot — unify the odd one out behind an Adapter.
- **Solution Sprawl (43)** — paired with *Move Creation Knowledge to Factory (68)*: the code implementing one responsibility (here: object creation) has sprawled across multiple classes; gather it into a Factory.

Conversely, eight of Fowler's smells reappear in Kerievsky's table marked [F] with *pattern-level* fixes: Alternative Classes with Different Interfaces (43), Duplicated Code (39), Large Class (44), Lazy Class (43), Long Method (40), Primitive Obsession (41), Switch Statements (44) (HighLevelRefactoring p.12–13). Fowler smells *not* re-listed by Kerievsky (Long Parameter List, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Parallel Inheritance Hierarchies, Speculative Generality, Temporary Field, Message Chains, Middle Man, Inappropriate Intimacy, Incomplete Library Class, Data Class, Refused Bequest, Comments) stay with their low-level Fowler fixes.

Kerievsky's table read backwards — refactoring → smells it resolves (HighLevelRefactoring p.12–13)

The smell→refactoring table is usually quizzed forwards ("name a fix for smell X"); reading it *backwards* exposes which composite refactorings are the multi-tools. Derived purely by inverting the table on the slides:

Composite refactoring	Resolves smell(s)
Unify Interfaces with Adapter (247)	Alternative Classes with Different Interfaces; Duplicated Code; Oddball Solution
Replace Conditional Logic with Strategy (129)	Conditional Complexity; Long Method; Primitive Obsession
Replace State-Altering Conditionals with State (166)	Conditional Complexity; Large Class; Primitive Obsession
Replace Implicit Language with Interpreter (269)	Combinatorial Explosion; Large Class; Primitive Obsession
Replace Conditional Dispatcher with Command (191)	Large Class; Long Method; Switch Statements
Introduce Null Object (301)	Conditional Complexity; Duplicated Code
Move Embellishment to Decorator (144)	Conditional Complexity; Primitive Obsession
Move Accumulation to Visitor (320)	Long Method; Switch Statements
Form Template Method (205)	Duplicated Code
Introduce Polymorphic Creation with Factory Method (88)	Duplicated Code
Chain Constructors (340)	Duplicated Code
Replace One/Many Distinctions with Composite (224)	Duplicated Code
Extract Composite (214)	Duplicated Code
Encapsulate Classes with Factory (80)	Indecent Exposure
Inline Singleton (114)	Lazy Class
Compose Method (123)	Long Method
Move Accumulation to Collecting Parameter (313)	Long Method
Replace Type Code with Class (286)	Primitive Obsession
Replace Implicit Tree with Composite (178)	Primitive Obsession
Encapsulate Composite with Builder (96)	Primitive Obsession
Move Creation Knowledge to Factory (68)	Solution Sprawl

Patterns to notice: **Duplicated Code is the most-served smell** (seven distinct fixes — it is the most attacked problem in the whole catalog); the three big *conditional* refactorings (Strategy, State, Command) each serve three smells; and four refactorings are one-smell specialists whose names practically state their smell (Encapsulate Classes with Factory → Indecent Exposure; Inline Singleton → Lazy Class; Move Creation Knowledge to Factory → Solution Sprawl; Replace Type Code with Class → Primitive Obsession) (HighLevelRefactoring p.12–13).

One problem, three rulers — Fowler vs SIG vs Kerievsky on the same smells

The lecture's deepest compare/contrast: the same underlying problem appears in all three decks under different guises — Fowler names the *symptom* and a local move, SIG turns it into a *threshold*, Kerievsky offers a *pattern-level* resolution. Side by side (every cell cited from its deck):

Underlying problem	Fowler — smell & low-level fix (Refactoring1)	SIG — metric form (BetterCode)	Kerievsky — pattern fix (HighLevelRefactoring)
Method too long	Long Method → Extract Method (110), Replace Method with Method Object (135) (p.6, 11, 13)	G1: unit > 15 LOC → split (p.7, 10–11)	Compose Method (123); Move Accumulation to Collecting Parameter (313); Replace Conditional Dispatcher with Command (191); Move Accumulation to Visitor (320); Replace Conditional Logic with Strategy (129) (p.13)
Conditional logic too complex	Switch Statements → Replace Conditional with Polymorphism (255); also Decompose Conditional (238), Guard Clauses (250) (p.7, 32–35)	G2: > 4 branch points (CC > 5) → split/simplify (p.13, 16, 18)	Conditional Complexity → Strategy (129) / Decorator (144) / State (166) / Null Object (301); Switch Statements → Command (191) / Visitor (320) (p.12–13)
Code duplicated	Duplicated Code → Extract Method, Extract Superclass (336), Form Template Method (345), Pull Up Constructor Body (325), Parameterize Method (283) (p.6, 39, 43–48)	G3: duplicated block ≥ 6 lines → write code once (p.20, 23)	Form Template Method (205); Factory Method (88); Chain Constructors (340); Composite (224/214); Adapter (247); Null Object (301) (p.12)
Too many parameters	Long Parameter List / Data Clumps → Introduce Parameter Object (295), Replace Parameter with Method (292) (p.6, 40–41)	G4: > 4 parameters → extract parameters into objects (p.25, 28)	Extract Parameter (346) appears as a helper move (Study Sequence session 18) (p.15)
Class doing too much	Large Class → Extract Class (149), Extract Subclass (330), Extract Interface (341) (p.6, 17, 44–45)	Module-level concern → G5 separate concerns / fan-in bands (p.31, 34) [course-context link]	Large Class → Command (191) / State (166) / Interpreter (269) (p.12)

The exam-grade takeaway: given any of these problems, you should be able to answer in *all three vocabularies* — name the smell and a Fowler move, state the SIG threshold it breaks, and name the Kerievsky pattern route. Answers that mix the vocabularies up (e.g. "Long Method violates G3") lose the easy marks.

JHotDraw Connection

JHotDraw is the course's running case study and the lab's target codebase. It matters here because it is where the abstract catalog meets real source: a mature, pattern-rich Java drawing framework on which you *practise* finding and fixing smells.

- **The lab is on JHotDraw directly:** "Find Code smells in JHotDraw based on your change request and SonarLint... Apply one or more suitable Refactoring Patterns to get rid of the bad code smells" (RefactLab p.1). So Lecture 4's smells/refactorings are exercised *on JHotDraw's actual source*, not toy examples — you locate a smell tied to a real change request and remove it with a catalog move.
- **JHotDraw is a GoF-pattern showcase**, which makes Kerievsky's *refactoring-to-patterns* deck directly applicable: because the codebase already embodies many patterns, refactoring *toward* those patterns has clear targets. JHotDraw heavily uses **Factory Method** (`createFigure`), **Strategy** (handles/tools), **Composite** (`CompositeFigure` containing `Figure` s), **Observer** (figures notifying views), **Decorator** (figure decorations), **State/Command** (tools, undoable commands), **Template**

Method, and **Adapter** [course-context inference; these are JHotDraw's well-known pattern roster, aligning with Kerievsky's catalog targets].

- **Big refactorings map onto JHotDraw's architecture:** *Separate Domain from Presentation* and *Duplicate Observed Data* (189) are exactly the GUI/domain separation JHotDraw's MVC-style design embodies — the `Drawing` / `Figure` domain model vs the `DrawingView` / GUI presentation (Refactoring1 p.27, p.56). *Replace Conditional with Polymorphism* / *Extract Hierarchy* mirror how a `Figure` hierarchy replaces type-switching over shape kinds: instead of a switch on "is this a rectangle/ellipse/line", each shape is its own `Figure` subclass.
- **Process placement:** in a JHotDraw change request (e.g. "add a new figure type"), Prefactoring cleans the `Figure` hierarchy or a `Tool` to make the addition easy; Actualization adds the new `Figure` / `Tool` ; Postfactoring removes any duplication introduced (e.g. extract a shared `AbstractFigure` method) [course-context inference]. This is the whole change process in miniature on one codebase.
- **SonarLint** is the lab's smell detector — *what it is:* an IDE plugin that statically analyses code and flags issues live as you type. *What it's for here:* it is the automated analog of "Symptoms of Bad Code" and BetterCode's metrics, run inside the IDE against JHotDraw, so the lab's "find smells" step is tool-assisted rather than purely by eye (RefactLab p.1).

The lab handout in full — objectives, classwork, portfolio (RefactLab p.1)

The one-page handout has four parts; every detail below is on the sheet:

Introduction (verbatim core): "Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior. Its heart is a series of small behavior preserving transformations. Each transformation (called a 'refactoring') does little, but a sequence of these transformations can produce a significant restructuring. Since each refactoring is small, it's less likely to go wrong. The system is kept fully working after each refactoring, reducing the chances that a system can get seriously broken during the restructuring." (RefactLab p.1) — note the three safety claims packed in here: small steps are *less likely to go wrong*, the system is *kept fully working after each refactoring*, and this *reduces the chance of seriously breaking* the system mid-restructure.

Objectives (two): 1. Identify and understand Bad Code Smells. 2. Apply refactorings to get rid of Bad Code. (RefactLab p.1)

Classwork (the exact workflow): 1. Make sure you have your own feature branch, using `git checkout -b your-feature-development` — i.e. the feature branch is cut from the `development` branch, not from `main` (RefactLab p.1). 2. Follow the **feature branch workflow** (the handout cites [GitHub flow]) (RefactLab p.1). 3. **Install [sonarlint]** (RefactLab p.1). 4. **Find code smells in JHotDraw** based on (a) *your change request* and (b) *SonarLint* — the handout adds, dryly, "(shouldn't be a problem)", i.e. JHotDraw has plenty of smells to find (RefactLab p.1). 5. **Apply one or more suitable Refactoring Patterns** to get rid of the bad code smells; a footnote points to the online catalog: "See <http://refactoring.com/catalog/>" (RefactLab p.1).

Portfolio Work (what you must write up): - Describe the **code smell that triggered your refactoring** — the handout points you to **Chapter 4 in [Ker05]** (Kerievsky's code-smells chapter, the source of the smell→refactoring table in the HighLevelRefactoring deck) (RefactLab p.1). - Describe **what you plan to change** by refactoring (RefactLab p.1). - Describe the **strategy** of the refactorings (RefactLab p.1). - State **which of the refactorings from [Ker05] you applied and what the reasoning behind it was** (RefactLab p.1). - "Remember to describe the strategies and purpose of the Refactorings." (RefactLab p.1)

Two details that distinguish a precise answer from a vague one: the smells are found **relative to a change request** (you are not smell-hunting at random — the lab simulates Prefactoring for a concrete change), and

the refactorings you apply and justify are **Kerievsky's** ([Ker05] , the high-level catalog), with refactoring.com as the reference catalog — so the lab explicitly exercises the *composite* layer on top of SonarLint's automated smell detection.

Worked Example / Process Walkthrough

This section shows the *measure-then-refactor* loop on concrete code from the decks, so the thresholds stop being abstract numbers and become "this unit, this count, this verdict."

The BetterCode deck carries a recurring concrete example, `clearAllMatches` / `getVariableToTypeMapping`, that walks the loop. Combining the slides into one end-to-end story:

Before (the measured unit) — `clearAllMatches(String input, String word)` "Replaces all occurrences of a word in a given string with blanks." Measured against the guidelines (BetterCode p.10, 16, 28): - **G1 (length)**: 15 lines of code → exactly at the threshold (BetterCode p.10) — passing, but with no margin, so any growth breaks it. - **G2 (complexity)**: a `while` and a `for` = 2 branch points, + 1 = **cyclomatic complexity 3** → within the ≤ 4 -branch-point limit (BetterCode p.16). - **G4 (interface)**: `String input, String word` = **2 parameters** → within the ≤ 4 limit (BetterCode p.28).

The G3 duplication example (before → after) (BetterCode p.23): `getVariableToTypeMapping(...)` has **15 LOC, 2 parameters, 1 branch point, and 10 lines of code duplicated** with `getParameterTypes(...)`. *Why it's flagged*: the duplicated block exceeds the **≥ 6 -line** threshold, so it violates "Write Code Once." **After** refactoring, the duplicated logic is factored out (the second method shrinks to **15 LOC, 1 parameter, 1 branch point** by calling `functionHeader.copy()` and the shared helper) → duplication removed. *What was applied*: this is **Extract Method** (pull the shared block into one helper) + **Introduce Parameter Object** (collapse the related arguments) in action, verified by re-measuring — the numbers drop back under threshold, proving the fix.

The G2 example (complexity comparison) (BetterCode p.17): `doAutoCompleteLabel(...)` has **3 branch points**; `isMultiPartForm(...)` has **4 branch points** — both still pass (≤ 4), illustrating exactly where the line sits (the second is at the limit). The G4 version of the same two methods shows each has **1 parameter** (BetterCode p.29). *The teaching point*: the same unit is scored on several guidelines at once, and "passing" can mean "right at the edge."

The high-level analog (HighLevelRefactoring): the deck's "Hello World — Strategy / Factory / Main" example (p.3–5) shows the *same Hello World* implemented three ways to make the point that "There Are Many Ways To Implement A Pattern" (p.18–19) — there is no one true Strategy. The Jefferson→Franklin revision of the Declaration of Independence ("self-evident" replacing "sacred and undeniable") is the metaphor for *iterative behaviour-preserving improvement of expression* (p.6): the meaning is preserved while the wording is refined, exactly as refactoring preserves behaviour while refining structure. *The lesson*: refactoring *toward* a pattern (e.g. Strategy) is itself a sequence of small steps, and you choose among several valid endpoints.

The disciplined loop the whole lecture teaches: **measure → identify smell → pick refactoring from the catalog → apply in small behaviour-preserving steps → re-run tests → re-measure**. Repeat until the unit is under threshold. *Why this loop*: measuring tells you *where* and *when*, the catalog tells you *what move*, small steps + tests keep it *safe*, and re-measuring *proves* the smell is gone — closing the loop between Fowler (the moves), Kerievsky (the targets), and SIG (the ruler).

clearAllMatches in full — one unit, three guidelines (BetterCode p.10, 16, 28)

The deck's recurring measured unit, reconstructed from the rendered listing (BetterCode p.10):

```
/**
 * Replaces all occurrences of a word in a given string with blanks.
 */
public static String clearAllMatches(String input, String word) {
    String result = "";
    int startIndex = 0;
    int matchIndex = input.indexOf(word);

    while (matchIndex >= 0) {
        result += input.substring(startIndex, matchIndex);
        // Insert spaces as filler
        for (int i = 0; i < word.length(); i++) {
            result += " ";
        }
        startIndex = matchIndex + word.length();
        matchIndex = input.indexOf(word, startIndex);
    }

    result += input.substring(startIndex);
    return result;
}
```

How the three measure slides score this *same* unit: - **G1 — count lines of code**: "Every line in the unit that is non-empty and does not contain only comments is a line of code." The listing spans 21 physical lines, but blank lines and the Javadoc `// Insert spaces as filler` comment lines don't count → **15 lines of code**, bang on the limit (BetterCode p.10). - **G2 — count cyclomatic complexity**: "Every branch point (if, case, for, &&, ||) is counted, and we add 1 to the total." The slide circles the `while` and the `for` → **2 branch points + 1 = cyclomatic complexity of 3** (BetterCode p.16). Note what does *not* count here: the assignments, the method calls, the `return`. - **G4 — count parameters**: "Every parameter in the unit header." The slide highlights `String input` and `String word` → **2 parameters** (BetterCode p.28).

The pedagogical point of reusing one listing: a unit gets a *vector* of scores (15 LOC, CC 3, 2 params), each judged against its own threshold — maintainability is multi-dimensional even at the single-method level.

The G2/G4 example pair — doAutoCompleteLabel and isMultiPartForm (BetterCode p.17, 29)

The example slides for G2 and G4 also reuse one pair of real-world units (the left one is Jenkins-flavoured Java — `Jenkins.getInstance()`, `AutoCompletionCandidates`):

```
public AutoCompletionCandidates doAutoCompleteLabel(@QueryParameter String value) {
    AutoCompletionCandidates c = new AutoCompletionCandidates();
    Set<Label> labels = Jenkins.getInstance().getLabels();
    List<String> queries = new AutoCompleteSeeder(value).getSeeds();

    for (String term : queries) {
        for (Label l : labels) {
            if (l.getName().startsWith(term)) {
                c.add(l.getName());
            }
        }
    }
    return c;
}
```



```

public static boolean isMultiPartForm(@CheckForNull String contentType) {
    if (contentType == null) {
        return false;
    }

    String[] parts = contentType.split(";");
    if (parts.length == 0) {
        return false;
    }

    for (int i = 0; i < parts.length; i++) {
        if ("multipart/form-data".equals(parts[i])) {
            return true;
        }
    }
    return false;
}

```

- **G2 verdict (BetterCode p.17):** `doAutoCompleteLabel` has **3 branch points** (the two nested `for`s and the `if`); `isMultiPartForm` has **4 branch points** (three `if`s and one `for`). Both pass the ≤ 4 limit — but the second sits *exactly at* the threshold: one more `if` and it must be split.
- **G4 verdict (BetterCode p.29):** each unit has **1 parameter** (`value`; `contentType`) — comfortably under the ≤ 4 limit. Annotation styles to notice: `@QueryParameter` and `@CheckForNull` annotations do not add parameters; the count is of parameters in the unit header.

The G3 duplication slide, read precisely (BetterCode p.23)

The G3 measure slide shows two side-by-side units and stamps a "WRONG WAY — GO BACK" sign between them with the caption "**Duplicated code blocks ≥ 6 lines of code**". Reading the annotations exactly as printed:

- Left unit — `public Map<String, String> getVariableToTypeMapping(ScopeContainer scopeContainer, INode node)` (an `@Override`): annotated "**This unit has 15 lines of code, 2 parameters, 1 branch point and 10 lines of code duplicated.**"
- Right unit — `private Map<String, String> getParameterTypes(TokenIterator functionHeader):` annotated "**This unit has 15 lines of code, 1 parameter, 1 branch point and 10 lines of code duplicated.**"

Both bodies share the same 10-line core (create a `Matcher` over the tokens, `setExpression(TYPE_DECLARATION)`, `setFindShortestMatch(false)`, then a `while (p != null)` loop pulling `NAME / TYPENAME` tokens into the result map via `result.put(variableNameToken.toString(), variableTypeToken.toString())` and `p = matcher.findNext()`); they differ in how they obtain the token stream (`getInterestingTokens(scopeContainer)` vs `functionHeader.copy()`). **Precision note:** the slide presents *both* units as currently containing the 10 duplicated lines — i.e. it is a *violation exhibit* ($10 \geq 6 \rightarrow G3$ flags it), and the refactoring direction (extract the shared block so it exists once) is what the guideline demands; an earlier paragraph in this guide reads the pair as a before/after, so treat the exact annotations above as the authoritative slide content (BetterCode p.23).

Also notice what the annotations demonstrate: G3's detector is *textual block duplication* with a hard size floor (≥ 6 lines), and a unit can pass G1 (15 LOC), G2 (1 branch point) and G4 (1–2 params) while still failing G3 — the guidelines are independent axes.

The G5, G6 and G7 measure visuals (BetterCode p.34, 39–40, 45–46)

- **G5 (BetterCode p.34):** header — "Count unit dependencies between modules: module coupling (fan-in at file level): every dependency between **units outside the module (A)** and **units inside the module (B)** increase coupling between modules A and B; module size (measured in lines of code): tightly coupled larger modules are bigger maintenance problems than smaller modules." The diagram shows modules B, C and D each drawing a **use**-arrow into module A → "**Fan-in = 3**". Beside it, the threshold table: Module coupling (fan-in at file level) **1–10** → **Excellent**, **11–20** → **Good**, **21–50** → **Difficult**, **>50** → **Impossible** (the right column is headed "Modifiability/Testability").
- **G6 (BetterCode p.39):** two counting steps. *Dependencies*: "count dependencies and cyclic dependencies among components." *The weighted percentage of hidden code*: "count those modules that **contain no external calls** or only calls to other modules; count **interface code**, viz. those modules that have **incoming calls**, or both **incoming and outgoing calls (throughput)**; **weigh** the hidden code and the interface code — the percentage of hidden code determines the component independence."
- **G6 example (BetterCode p.40):** two honeycomb diagrams of a component made of 12 modules (hexagons). Left, captioned "Loosely coupled components: easy to change in isolation — few external dependencies, code responsibilities separated, easy to test": almost all hexagons are *internal* (one colour), with only a couple of interface modules at the edge. Right — "many external dependencies, multiple code responsibilities, hard to test": many hexagons are coloured as outgoing/incoming/throughput modules with dependency arrows sprouting on all sides. The legend names the module roles: **Internal, Outgoing, Incoming, Throughput** (plus "Component with 12 modules" and "Dependency From-To").
- **G7 measure (BetterCode p.45):** "A combined calculation of: (1) number of top-level components; (2) the uniformity of component size."
- **G7 example (BetterCode p.46):** two bubble charts. Left: one enormous bubble ringed by dozens of tiny ones — the unbalanced anti-example (one god component plus crumbs). Right: a cluster of bubbles of roughly equal diameter — the balanced target. No numbers on the slide; the shape difference is the lesson.

Definitions & Terminology

Each row below: *what the term means, why it matters / what it's used for*, and where relevant *how it's applied*.

Term	Definition	Source
Refactoring	A behaviour-preserving change to internal structure to make code easier to understand and cheaper to modify. <i>Used to</i> : repay design debt and keep a system cheap to keep changing, without altering what it does.	Refactoring 1 p.3
Behaviour preservation	The hard constraint that observable behaviour (same inputs → same outputs/side-effects) must not change. <i>Why</i> : it is what makes refactoring <i>safe</i> and <i>verifiable</i> (by tests/tools); break it and it is a feature change, not a refactoring.	Refactoring 1 p.3; RefactLab p.1
Prefactoring	Refactoring <i>before</i> Actualization to make a planned change easy. <i>Used to</i> : reshape the change region so the new behaviour has a clean place to live (Fowler's "refactor when you add function"); adds no functionality.	course; Refactoring 1 p.5
Postfactoring	Refactoring <i>after</i> Actualization to remove smells the change introduced. <i>Used to</i> : leave the codebase no worse than found (Boy-Scout rule) before concluding; the discipline behind SIG G10.	course; BetterCode p.57

Term	Definition	Source
Code smell / Symptom of Bad Code	A surface indication that usually corresponds to a deeper problem and a candidate refactoring. <i>Used as:</i> a trigger language — naming the smell points you to the fix. <i>Note:</i> a smell is not a bug.	Refactoring 1 p.6
Rule of Three	Heuristic: refactor the third time you do something similar. <i>Used to:</i> time the removal of duplication — avoid premature abstraction (1st), tolerate one duplicate (2nd), abstract once the pattern is real (3rd).	Refactoring 1 p.5
Cyclomatic complexity	branch points + 1 (<code>if</code> / <code>case</code> / <code>for</code> / <code>&&</code> / <code> </code> each count as 1). <i>Used to:</i> measure the number of independent paths through a unit $\approx$ the minimum number of tests needed; the basis of SIG G2.	BetterCode p.16
Branch point	An <code>if</code> , <code>case</code> , <code>for</code> , <code>&&</code> , or <code> </code> — each a place where control flow forks. <i>Used to:</i> count complexity for G2 (≤ 4 branch points per unit).	BetterCode p.16
Unit / Module / Component / System	SIG levels: method / file-class / package-dir / whole codebase. <i>Used to:</i> fix the granularity at which each guideline is measured (complexity per unit, coupling per module, balance per component, size per system).	BetterCode p.9
Fan-in (module coupling)	The count of dependencies from units <i>outside</i> a module into units <i>inside</i> it. <i>Used to:</i> measure G5 module coupling — high fan-in means many things break when the module changes (1–10 excellent ... >50 impossible).	BetterCode p.34
Hidden code	Modules with no external calls (or only outgoing calls); the % hidden = component independence. <i>Used to:</i> measure G6 — the more code a component hides behind a thin interface, the more loosely coupled and independently replaceable it is.	BetterCode p.39
Man-months (size)	LOC $\div$ average developer productivity (LOC/year) — person-months to build the system. <i>Used to:</i> gauge codebase size for G8, since large size predicts higher defect density and project-failure risk.	BetterCode p.51
Composite / high-level refactoring	A sequence of low-level refactorings that introduces a design pattern. <i>Used to:</i> solve <i>design</i> (not just local) problems gradually, with an overall plan toward a known pattern.	HighLevelRefactoring p.16
Algebra of refactoring	Solving design problems via behaviour-preserving steps (patterns = word problems, refactoring = algebra). <i>Used to:</i> frame design as something you <i>reach</i> by composing small moves, generalising beyond the catalog's listed problems.	HighLevelRefactoring p.8–10
Automation First	Prefer tool-automated refactorings ("highways") over manual ones ("dirt roads"). <i>Used to:</i> refactor faster and more safely by letting the IDE preserve behaviour mechanically.	HighLevelRefactoring p.20
Client First	Start refactoring from a client of the smelly code to find a simpler path. <i>Used to:</i> uncover an easier, often tool-supported route into the target structure by changing how callers use the code.	HighLevelRefactoring p.20

Code smells (22) — *Duplicated Code, Long Method, Large Class, Long Parameter List, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Switch Statements, Parallel Inheritance Hierarchies, Lazy Class, Speculative Generality, Temporary Field, Message Chains, Middle Man, Inappropriate Intimacy, Alternative Classes with Different Interfaces, Incomplete Library Class, Data Class, Refused Bequest, Comments* (Refactoring1 p.6–8). **Kerievsky-only smells:** *Combinatorial Explosion, Conditional Complexity, Indecent Exposure, Oddball Solution, Solution Sprawl* (HighLevelRefactoring p.12–13).

Refactorings (Fowler catalog, by category) — *Extract Method, Inline Method, Replace Method with Method Object* (Composing); *Move Method, Move Field, Extract Class, Inline Class, Hide Delegate, Remove Middle Man, Introduce Foreign Method, Introduce Local Extension* (Moving Features); *Self Encapsulate*

Field, Replace Data Value with Object, Replace Array with Object, Duplicate Observed Data, Encapsulate Field, Encapsulate Collection, Replace Subclass with Fields (Organizing Data); Decompose Conditional, Consolidate Conditional Expression, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism, Introduce Null Object (Conditionals); Separate Query from Modifier, Parameterize Method, Replace Parameter with Method, Introduce Parameter Object (Method Calls); Pull Up Constructor Body, Extract Subclass, Extract Superclass, Extract Interface, Collapse Hierarchy, Form Template Method, Replace Inheritance with Delegation, Replace Delegation with Inheritance (Generalization); Tease Apart Inheritance, Convert Procedural Design to Objects, Separate Domain from Presentation, Extract Hierarchy (Big) (Refactoring1 p.11–58).

Additional terminology from the rendered slides

Term	Definition	Source
Pattern (Alexander)	"A three-part rule, which expresses a relation between a certain context , a problem , and a solution "; simultaneously "a process and a thing." <i>Used to:</i> define what Kerievsky's composite refactorings aim at — the "process" half is what a refactoring sequence walks.	HighLevelRefactoring p.7
Structure diagram (of a pattern)	"Just an example, not a specification" — it portrays the most common implementation; differences are "inevitable and actually desirable" (Vlissides). <i>Used to:</i> defeat the trap that code must match the GoF diagram to count as the pattern.	HighLevelRefactoring p.18–19
To / Towards / Away (refactoring directions)	The three direction columns of Kerievsky's matrix: a move that takes you fully <i>to</i> a pattern, <i>toward</i> it (partial progress), or <i>away</i> from it (removing an over-applied pattern).	HighLevelRefactoring p.21–22
Unit (in Java)	Method or constructor — the smallest grouping of lines executable independently.	BetterCode p.4
Module (in Java)	Top-level class, interface, or enum — the smallest grouping of units.	BetterCode p.4
Component / System (in Java)	Not defined by the language — they exist only in the architecture (top-level directory, package, namespace, Maven sub-module, VS solution for components; the whole codebase for the system).	BetterCode p.4, 9
Lines of code (G1 counting rule)	"Every line in the unit that is non-empty and does not contain only comments is a line of code" — blanks and comment-only lines are excluded.	BetterCode p.10
Interface module (G6)	A module with incoming calls from other components (or incoming + outgoing = throughput); the opposite of hidden code. Module roles in the example legend: Internal, Outgoing, Incoming, Throughput.	BetterCode p.39–40
Duplicated block (G3)	A block of ≥ 6 lines of code occurring more than once — the hard floor of the G3 detector ("WRONG WAY GO BACK").	BetterCode p.23
Feature branch (lab)	Created with <code>git checkout -b your-feature development</code> — cut from the <code>development</code> branch, worked under the [GitHub flow] feature-branch workflow.	RefactLab p.1
SonarLint (lab)	The IDE static-analysis plugin the lab installs to find code smells in JHotDraw ("based on your change request and sonarlint").	RefactLab p.1
Tear Apart Inheritance	The slide title for the big refactoring Fowler's book names "Tease Apart Inheritance" — one hierarchy doing two jobs is split into two hierarchies joined by delegation.	Refactoring1 p.52–53

Kerievsky-only smells (5): *Combinatorial Explosion (45), Conditional Complexity (41), Indecent Exposure (42), Oddball Solution (45), Solution Sprawl (43)* — names and [Ker05] pages exactly as on the table slides (HighLevelRefactoring p.12–13).

Kerievsky refactorings appearing only in the Study Sequence (not in the smell table): *Replace Constructors with Creation Methods* (57), *Extract Adapter* (258), *Limit Instantiation with Singleton* (296), *Replace Hard-Coded Notifications with Observer* (236), *Unify Interfaces* (343), *Extract Parameter* (346) (HighLevelRefactoring p.14–15; Observer also appears in the Directions matrix p.22).

Common Pitfalls / Gotchas

- **"Refactoring changes behaviour" — WRONG.** By definition it does not. If observable behaviour changes, it is not a refactoring; it is a modification/feature change. This is the most common exam trap (Refactoring1 p.3).
- **Smell ≠ bug.** A smell is a *symptom*, not a defect; it doesn't always require action (Refactoring1 p.6). Don't claim a smell is necessarily a bug.
- **Inverse pairs exist** — Extract Method ↔ Inline Method; Extract Class ↔ Inline Class; Hide Delegate ↔ Remove Middle Man; Replace Inheritance with Delegation ↔ Replace Delegation with Inheritance; refactor *towards* ↔ *away* from a pattern. The right one depends on which smell dominates; **Middle Man** (too much delegation) vs **Message Chains/Inappropriate Intimacy** (too little) pull in opposite directions.
- **Divergent Change vs Shotgun Surgery** are mirror images — Divergent Change = *one class changes for many reasons*; Shotgun Surgery = *one change touches many classes*. Don't swap them.
- **Thresholds are precise — memorise them:** G1 = **15 LOC**, G2 = **≤4 branch points** (CC ≤5), G3 = **duplication block ≥6 lines**, G4 = **≤4 parameters**, G5 fan-in **1–10 excellent**, G7 = **~9 components (6–12), equal size**. Cyclomatic complexity = branch points +1 (don't forget the +1).
- **"Comments" is a smell**, but the fix is *not* "delete all comments" — it's Extract Method + a self-documenting name so the comment becomes unnecessary (Refactoring1 p.8).
- **Over-using patterns is a smell too.** Kerievsky's "Refactoring Directions" includes moves *away* from patterns (e.g. Inline Singleton). Speculative Generality is the Fowler-equivalent warning (Refactoring1 p.7; HighLevelRefactoring p.22).
- **No tests = unsafe refactoring.** Behaviour preservation must be *verified*; that's why G9 (Automated Tests) and the Verification phase exist. "I refactored without running tests" is wrong practice.
- **Book page numbers ≠ slide numbers.** The "(110)", "(255)" etc. are *[Fowler99]/[Kero5] page numbers* printed on the slides, not lecture slide numbers.
- **Prefactoring vs Postfactoring** — both behaviour-preserving; they differ only by *position* around Actualization. Don't conflate either with Actualization (which *does* add behaviour).
- **"Tear Apart" vs "Tease Apart" Inheritance.** The slides title it *Tear Apart Inheritance* (Refactoring1 p.52–53); Fowler's book calls it *Tease Apart Inheritance*. Same refactoring — recognise both names.
- **The p.5 overview rennumbers/renames the guidelines.** On the Guideline Levels slide, item 3 is "Keep Interfaces Small" and item 4 is "Write Code Once" — the *reverse* of the chapter numbering (G3 = Write Code Once, G4 = Keep Unit Interfaces Small), and titles are shortened ("Keep Units Short", "Automate Tests") (BetterCode p.5 vs p.19–29). Check which numbering a question uses.
- **A unit is a method or constructor; a module is a top-level class, interface or enum; component and system have no Java construct** (BetterCode p.4). Saying "module = package" confuses module with component.
- **G1's line counting excludes blanks and comment-only lines** — the `clearAllMatches` listing spans 21 physical lines but counts 15 LOC (BetterCode p.10). Counting physical lines is the classic mistake.

- **G2 counts `&&` and `||` as branch points**, not just `if / case / for` — a one-`if` condition with two `&&`s already has 3 branch points (BetterCode p.16).
- **A unit can pass G1, G2 and G4 and still fail G3** — the p.23 exhibit units have 15 LOC, 1 branch point, 1–2 parameters, *and* 10 duplicated lines. The guidelines are independent axes (BetterCode p.23).
- **The Directions matrix has THREE direction columns — To, Towards, Away** — not two. "To" completes the pattern, "Towards" makes partial progress, "Away" retreats from it (HighLevelRefactoring p.21–22). The Iterator row is Away-only; the Singleton row spans To and Away.
- **A pattern's Structure diagram is "just an example, not a specification"** (Vlissides) — deviating from the GoF diagram does not disqualify an implementation (HighLevelRefactoring p.18–19).
- **The patterns-happy Hello World is a warning, not a model** — Strategy + Factory + Singleton + anonymous class to print one string; be able to *name* the patterns in it and *diagnose* it (Speculative Generality / over-engineering) (HighLevelRefactoring p.3–5; Refactoring1 p.7).
- **Lab branch is cut from development** — `git checkout -b your-feature development`, not from `main/master` (RefactLab p.1). And the portfolio's smell vocabulary comes from **Chapter 4 of [Kero5]**, not Fowler.
- **Slide 21 of Refactoring1 holds two refactorings** — Introduce Foreign Method *and* Introduce Local Extension share the slide; the (162)/(164) book pages distinguish them. One missing method → Foreign Method; several → Local Extension.
- **Inline Method's example is not Extract Method backwards on the same code** — the deck uses `printOwing` for Extract (p.11) but `getRating / moreThanFiveLateDeliveries` for Inline (p.12). But Extract Class (p.17) and Inline Class (p.18) *do* share one Person/Telephone Number example — recognise reversed diagrams.
- **G6 exposure is about receiving calls, not making them.** The rule's "how" clause defines exposed code as code "exposed to (i.e., **can receive calls from**) modules in other components" — a module that only makes outgoing calls still counts as hidden (BetterCode p.36, 39).

Exam Focus

- **Define refactoring** in one sentence and state the behaviour-preservation constraint (Refactoring1 p.3). Almost guaranteed.
- **Place Prefactoring and Postfactoring** in the eight-phase change process and explain the purpose of each.
- **Reproduce the SIG thresholds** cold: 15 LOC / 4 branch points / 6-line duplication / 4 parameters / fan-in bands / ~9 components. Map guideline → level (Unit/Module/Component/System).
- **Match smell → refactoring**: given a smell (Long Method, Duplicated Code, Switch Statements, Feature Envy, Long Parameter List, Data Clumps), name the right Fowler refactoring (Extract Method; Extract Superclass/Form Template Method; Replace Conditional with Polymorphism; Move Method; Introduce Parameter Object). Be able to do the Kerievsky pattern version too (Switch → Replace Conditional Dispatcher with Command / Replace Conditional Logic with Strategy).
- **Explain high-level vs low-level refactoring** and "refactoring to patterns"; quote "patterns are the word problems, refactoring is the algebra" and explain composite refactorings' three benefits (overall plan, non-obvious directions, pattern insight).
- **Cyclomatic complexity calculation**: given a code snippet, count branch points and add 1; state whether it passes G2.
- **JHotDraw + lab**: describe the lab workflow (feature branch → SonarLint → find smells → apply Kerievsky refactoring → document strategy) and name patterns JHotDraw uses that refactoring targets.

- **Inverse pairs & mirror smells:** be ready to distinguish Divergent Change/Shotgun Surgery and Middle Man/Message Chains, and to give the inverse of a named refactoring.
- **Reproduce a slide example for a named refactoring:** `printOwing` / `printDetails` (Extract Method), `getRating` (Inline Method), `Order` / `PriceCalculator` (Method Object), `Person`/Telephone Number (Extract↔Inline Class), `Person`/Department (Hide Delegate↔Remove Middle Man), `Date` / `MfDate` (Local Extension), `getSpeed` birds (Replace Conditional with Polymorphism), `getPayAmount` (Guard Clauses), `disabilityAmount` (Consolidate Conditional), Site billing (Form Template Method), `Stack` extends `Vector` (Replace Inheritance with Delegation), `Deal`/Presentation Style (Tear Apart Inheritance), Billing Scheme (Extract Hierarchy) (Refactoring1 p.11–58).
- **Quote a Rule–How–Why triple:** given a "This improves maintainability because..." clause, name the guideline (e.g. "clean code is maintainable code" → G10; "small product, project, and team is a success factor" → G8; "independent components ease isolated maintenance" → G6) (BetterCode rule slides).
- **The SIG Concepts table:** unit = method *or* constructor; module = top-level class/interface/enum; component & system = not defined by the language (BetterCode p.4).
- **Count LOC/branch points/parameters on a listing:** `clearAllMatches` = 15 LOC, 2 branch points (CC 3), 2 parameters; `doAutoCompleteLabel` = 3 branch points, 1 parameter; `isMultiPartForm` = 4 branch points (at the limit), 1 parameter (BetterCode p.10, 16–17, 28–29).
- **Alexander's definition:** a pattern is a three-part rule — context, problem, solution — and is "both a process and a thing" (HighLevelRefactoring p.7).
- **Name the patterns in the Hello World exhibit** (Strategy, factory machinery, Singleton, anonymous class) and explain what "Patterns Happy?" is asking (HighLevelRefactoring p.3–5).
- **Navigate the Directions matrix:** give the To/Towards/Away move for a named pattern — especially Singleton (Limit Instantiation vs Inline Singleton), Composite (three To-moves; Builder as Away), Iterator (Away-only) (HighLevelRefactoring p.21–22).
- **Study Sequence ordering logic:** creational first (sessions 1–3), Strategy/Template Method next (4–5), Visitor and Interpreter last (20–21) (HighLevelRefactoring p.14–15).
- **Lab specifics:** branch from `development`, GitHub flow, SonarLint, smells tied to *your change request*, refactorings and smell descriptions from [Kero5] (Chapter 4), catalog reference refactoring.com/catalog (RefactLab p.1).

Rapid-recall checklist (self-test, answers in parentheses)

Run through these cold before the exam; every answer is a slide fact cited earlier in this guide.

- Refactoring changes the ___ **structure**; the _____ behavior must not change. (internal; observable) (Refactoring1 p.3)
- The four "why refactor" reasons? (improves design; easier to understand; helps find bugs; helps you program faster) (Refactoring1 p.4)
- The four "when refactor" triggers? (Rule of Three; when you add function; when you fix a bug; as you do a code review) (Refactoring1 p.5)
- How many code smells in Fowler's "Symptoms of Bad Code"? (22, over three slides) (Refactoring1 p.6–8)
- How many refactoring categories, and the odd one out? (7; Big Refactorings — large-scale, multi-step) (Refactoring1 p.9)
- G1 / G2 / G3 / G4 thresholds? (15 LOC; ≤4 branch points; no duplicated blocks ≥6 lines; ≤4 parameters) (BetterCode p.7, 13, 20–23, 25)
- What counts as a branch point? (if, case, for, &&, ||; CC = branch points + 1) (BetterCode p.16)
- G5 fan-in bands? (1–10 excellent, 11–20 good, 21–50 difficult, >50 impossible) (BetterCode p.34)

- G6 module roles in the example legend? (Internal, Outgoing, Incoming, Throughput; hidden-code % = component independence) (BetterCode p.39–40)
- G7 target? (≈ 9 components, between 6 and 12, approximately equal size) (BetterCode p.42)
- G8 size measure? (man-months = LOC $\div$ average developer productivity in LOC/year) (BetterCode p.51)
- G9's five motivation points? (repeatable testing; efficient development; predictable code; tests document the code; writing tests makes you write better code) (BetterCode p.54)
- G10's seven "leave no ... behind" items? (unit-level code smells; bad comments; code in comments; dead code; long identifier names; magic constants; badly handled exceptions) (BetterCode p.58)
- Unit / Module in Java? (method or constructor; top-level class, interface, or enum) (BetterCode p.4)
- Kerievsky's five own smells? (Combinatorial Explosion; Conditional Complexity; Indecent Exposure; Oddball Solution; Solution Sprawl) (HighLevelRefactoring p.12–13)
- The two refactoring heuristics? (Automation First — highways over dirt roads; Client First — start from a client of the smelly code) (HighLevelRefactoring p.20)
- The three benefits of composite refactorings? (overall plan; non-obvious design directions; insights into implementing patterns) (HighLevelRefactoring p.16)
- The three columns of the Directions matrix? (To; Towards; Away) (HighLevelRefactoring p.21–22)
- How many study-sequence sessions? (21 — creational first, Visitor/Interpreter last) (HighLevelRefactoring p.14–15)
- The away-from-Singleton move? (Inline Singleton (114)) (HighLevelRefactoring p.22)
- Pattern definition's three parts? (context, problem, solution — Alexander) (HighLevelRefactoring p.7)
- Lab branch command? (`git checkout -b your-feature development`) (RefactLab p.1)
- Lab smell-chapter reference? (Chapter 4 in [Ker05]) (RefactLab p.1)

Source Map

Deck	Pag es	Sections / Content
Refactoring1.p df	3	Definition of refactoring (behaviour-preserving)
	4	Why refactor? (4 reasons)
	5	When refactor? (Rule of Three + 3 triggers)
	6–8	The 22 Code Smells (Symptoms of Bad Code, sets 1–3)
	9	The 7 refactoring categories
	10– 13	Composing Methods (Extract/Inline Method, Method Object)
	14– 22	Moving Features Between Objects (Move Method/Field, Extract/Inline Class, Hide Delegate, Remove Middle Man, Introduce Foreign Method/Local Extension)
	23– 30	Organizing Data (Self Encapsulate, Replace Data Value w/ Object, Replace Array w/ Object, Duplicate Observed Data, Encapsulate Field/Collection, Replace Subclass w/ Fields)
	31– 36	Simplifying Conditionals (Decompose, Consolidate, Guard Clauses, Polymorphism, Null Object)
	37– 41	Making Method Calls Simpler (Separate Query/Modifier, Parameterize, Replace Param w/ Method, Introduce Parameter Object)

Deck	Pag es	Sections / Content
	42–50	Dealing with Generalization (Pull Up Constructor, Extract Sub/Superclass, Extract Interface, Collapse Hierarchy, Form Template Method, Replace Inheritance↔Delegation)
	51–58	Big Refactorings (Tease Apart Inheritance, Convert Procedural to Objects, Separate Domain from Presentation, Extract Hierarchy)
	59	Reference — [Fowler99]
BetterCode.pdf	3	Intro — 10 guidelines, Clean Code, measurable metrics (bettercodehub.com)
	5	Guideline Levels overview (1–10 mapped to Unit/Module/Component/System)
	6–11	G1 Short Units — rule (15 LOC), motivation, level, measure, refactorings
	12–18	G2 Simple Units — rule (≤ 4 branch points), CC measure, example, Replace Conditional w/ Polymorphism
	19–23	G3 Write Code Once — rule (no copy), motivation, measure (≥ 6 -line duplication)
	24–29	G4 Small Unit Interfaces — rule (≤ 4 params), measure, example
	30–34	G5 Separate Concerns in Modules — fan-in coupling table (1–10/11–20/21–50/ > 50)
	35–40	G6 Couple Components Loosely — hidden-code %, dependencies, example
	41–46	G7 Balanced Components — ~ 9 components (6–12), equal size, measure, example
	47–51	G8 Small Codebase — defect density, man-months measure
	52–55	G9 Automated Tests — motivation (5 points), System level
	56–58	G10 Write Clean Code — leave no smells; 7 "How To" rules
HighLevelRefactoring.pdf	2	Kerievsky — Refactoring to Patterns (high-level catalog)
	3–5	Hello World implemented as Strategy / Factory / Main
	6–11	Pattern philosophy (Declaration of Independence revision; patterns = word problems, refactoring = algebra)
	12–13	Code Smell → Refactoring catalog (smell→pattern table)
	14–15	A Study Sequence (21 sessions ordering refactorings)
	16	Benefits of composite refactorings (3)
	17–19	Pattern: Factory Method structure; "many ways to implement a pattern"
	20	Patterns of Refactoring heuristics — Automation First, Client First

Deck	Pag es	Sections / Content
	21– 22	Refactoring Directions matrix (toward/away per GoF pattern)
RefactoringLab 1.pdf	1	Lab — feature branch, SonarLint, find smells in JHotDraw, apply [Ker05] refactorings, document strategy in portfolio

Figure index — which slide carries which example

For revision-by-figure: the example drawn on each information-bearing slide, so you can match a diagram or listing to its refactoring/guideline instantly.

Deck	Page	Figure / listing
Refactoring1	11	<code>printOwing</code> → <code>printDetails</code> code (Extract Method)
	12	<code>getRating</code> / <code>moreThanFiveLateDeliveries</code> code (Inline Method)
	13	<code>Order.price()</code> → <code>PriceCalculator</code> UML + <code>return new PriceCalculator(this).compute()</code>
	15–16	Generic <code>Class 1</code> / <code>Class 2</code> UML — <code>aMethod()</code> / <code>aField</code> migrating
	17–18	<code>Person</code> / <code>Telephone Number</code> UML (Extract Class; same diagram reversed for Inline Class)
	19–20	<code>Client</code> / <code>Person</code> / <code>Department</code> UML (Hide Delegate; reversed for Remove Middle Man)
	21	Text-only: both Introduce Foreign Method and Introduce Local Extension definitions
	22	<code>Client</code> <code>nextDay(Date):Date</code> → <code>MfDate</code> <code>extends Date</code> UML (Local Extension)
	24	<code>includes(int arg)</code> <code>_low</code> / <code>_high</code> → <code>getLow()</code> / <code>getHigh()</code> code (Self Encapsulate Field)
	25	<code>Order{customer:String}</code> → <code>Order</code> ♦→1 <code>Customer{name:String}</code> UML
	26	<code>String[] row</code> <code>Liverpool/15</code> → <code>Performance</code> object code (Replace Array with Object)
	27	<code>Interval Window</code> / <code>Interval</code> / <code>Observer</code> / <code>Observable</code> UML (Duplicate Observed Data)
	28	<code>public String _name</code> → private + <code>getName</code> / <code>setName</code> code (Encapsulate Field)
	29	<code>Person</code> <code>getCourses():Set/setCourses</code> → Unmodifiable Set + <code>addCourse</code> / <code>removeCourse</code> UML
	30	<code>Person</code> ← <code>Male/Female</code> <code>getCode()</code> returning 'M'/'F' → <code>code</code> field UML (Replace Subclass with Fields)
	32	Summer/winter <code>charge</code> conditional → <code>notSummer</code> / <code>winterCharge</code> / <code>summerCharge</code> code
	33	<code>disabilityAmount()</code> three guards → <code>isNotEligibleForDisability()</code> code
	34	<code>getPayAmount()</code> nested <code>_isDead</code> / <code>_isSeparated</code> / <code>_isRetired</code> → guard clauses code
	35	<code>getSpeed()</code> switch (<code>EUROPEAN</code> / <code>AFRICAN</code> / <code>NORWEGIAN_BLUE</code>) → Bird hierarchy UML
	36	<code>customer == null</code> → <code>Customer</code> / <code>Null Customer</code> UML (Introduce Null Object)
	38	<code>getTotalOutstandingAndSetReadyForSummaries</code> split UML (Separate Query from Modifier)

Deck	Page	Figure / listing
	39	<code>fivePercentRaise / tenPercentRaise</code> → <code>raise(percentage)</code> UML (Parameterize Method)
	40	<code>basePrice / discountLevel / discountedPrice</code> code (Replace Parameter with Method)
	41	<code>amountInvoicedIn(start,end) ×3</code> → <code>DateRange</code> UML (Introduce Parameter Object)
	43	<code>Manager extends Employee</code> constructor → <code>super(name, id)</code> code (Pull Up Constructor Body)
	45	<code>Employee</code> → «interface» <code>Billable {getRate, hasSpecialSkill}</code> UML (Extract Interface)
	46	<code>Employee</code> ← <code>Salesman</code> merged UML (Collapse Hierarchy)
	48	<code>Site</code> ← <code>Residential/Lifeline</code> <code>getBillableAmount</code> → <code>getBaseAmount()+getTaxAmount()</code> UML (Form Template Method)
	49	<code>Stack extends Vector</code> → <code>Stack</code> → 1 <code>Vector</code> , <code>return _vector.isEmpty()</code> UML
	50	<code>Employee</code> → 1 <code>Person</code> <code>return person.getName()</code> → <code>Employee</code> extends <code>Person</code> UML
	53	<code>Deal/Active/Passive × Tabular</code> → <code>Deal</code> + Presentation Style hierarchies UML (Tear Apart Inheritance)
	55	<code>Order Calculator</code> <code>determinePrice / determineTaxes</code> → <code>Order/Order Line</code> <code>getPrice / getTaxes</code> UML
	56	<code>Order Window</code> → <code>Order Window</code> → 1 <code>Order</code> UML (Separate Domain from Presentation)
	58	<code>Billing Scheme</code> → <code>Business/Residential/Disability</code> subclasses UML (Extract Hierarchy)
BetterCode	4	Concepts table: Unit/Module/Component/System with Java mapping
	5	Guideline Levels overview (shortened titles; items 3/4 swapped vs chapters)
	9, 15, 22, 33, 55 (etc.)	The reused nested Level diagram (Unit/Module/Component/System with examples)
	10, 16, 28	<code>clearAllMatches</code> listing measured for G1 (15 LOC), G2 (CC 3), G4 (2 params)
	17, 29	<code>doAutoCompleteLabel</code> (3 bp, 1 param) and <code>isMultiPartForm</code> (4 bp, 1 param)
	23	<code>getVariableToTypeMapping / getParameterTypes</code> duplication exhibit (10 lines duplicated each)
	34	Fan-in diagram (Fan-in = 3) + coupling table 1–10/11–20/21–50/>50
	39–40	Hidden-code counting rules + honeycomb component example (Internal/Outgoing/Incoming/Throughput)
	45–46	G7 combined calculation + bubble-chart balance example
	58	The seven "leave no ... behind" clean-code rules
HighLevelRefactoring	3–5	"Patterns Happy?" Hello World full code
	7	Alexander pattern definition [ATWoB p247]
	12–13	Code Smell → Refactoring table
	14–15	Study Sequence, sessions 1–21
	17–19	Factory Method structure; two alternative structures; Vlissides quote

Deck	Page	Figure / listing
	21–22	Refactoring Directions matrix (To / Towards / Away)

Slide-naming and coverage notes

- The deck title on Refactoring1 p.52–53 is "**Tear Apart Inheritance**"; the body of this guide (and Fowler's book) uses "Tease Apart Inheritance". They are the same big refactoring.
- BetterCode's per-guideline "Level" slides (p.9, 15, 22, 27, 33, 38, 44, 50, 55) all repeat one annotated nested diagram with the relevant level highlighted; the diagram's component examples are *top-level directory*, *package*, *namespace*, *Maven sub-module*, *Visual Studio solution*.
- Refactoring1's section-divider slides (p.2, 10, 14, 23, 31, 37, 42, 51) carry only the category title; p.59 is the [Fowler99] reference. BetterCode's chapter-cover slides (p.6, 12, 19, 24, 30, 35, 41, 47, 52, 56) repeat the guideline title before the Rule–How–Why slide.
- Book-page numbers in refactoring names — "(110)", "(345 a/b)", "(57)" — are [Fowler99]/[Kero5] page numbers printed on the slides; "345 a" and "345 b" mark the two Form Template Method slides (definition and figure) (Refactoring1 p.47–48).
- **Deck metadata:** all three lecture decks carry the same title-slide credit — "(SB5-MAI), Jan Corfixen Sørensen, University of Southern Denmark" (Refactoring1 p.1; BetterCode p.1; HighLevelRefactoring p.1) — and the lab handout is likewise authored by Jan Corfixen Sørensen (RefactLab p.1). Refactoring1's closing reference slide gives the full citation: "Fowler, M., *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999" (Refactoring1 p.59). HighLevelRefactoring's introduction slide names its source as "Refactoring to Patterns — High Level Catalog — Joshua Kerievsky" (HighLevelRefactoring p.2), and BetterCode's introduction announces "Software Maintenance in 10 Guidelines — Clean Code — Measurable Software Metrics (bettercodehub.com)" (BetterCode p.3).
- **Quote-attribution map for the high-level deck:** the algebra/word-problems quotes (p.8–10) and the "primer, not reference book" instruction (p.11) are from *Refactoring to Patterns* and its forewords; the pattern definition (p.7) is attributed on-slide to [Alexander, ATwOB, p247] ; the structure-diagram caveat (p.19) to [Vlissides, C++ Report, April 1998] . Knowing which authority backs which claim (Alexander → what a pattern is; Vlissides → diagrams are examples; Kerievsky → algebra of refactoring) is a cheap exam differentiator.